Community Detection with the Map Equation and Infomap: Theory and Applications

JELENA SMILJANIĆ, IceLab, Department of Physics, Umeå University, Umeå, Sweden and Scientific Computing Laboratory, Center for the Study of Complex Systems, Institute of Physics Belgrade, University of Belgrade, Belgrade, Serbia
CHRISTOPHER BLÖCKER, IceLab, Department of Physics, Umeå University, Umeå, Sweden, Center for Artificial Intelligence and Data Science, University of Würzburg, Würzburg, Germany, and Data Analytics Group, Department of Informatics, University of Zürich, Zürich, Switzerland
ANTON HOLMGREN, IceLab, Department of Physics, Umeå University, Umeå, Sweden
DANIEL EDLER, IceLab, Department of Physics, Umeå University, Umeå, Sweden and Gothenburg Global Biodiversity Centre, University of Gothenburg, Gothenburg, Sweden
MAGNUS NEUMAN, IceLab, Department of Physics, Umeå University, Umeå, Sweden
MARTIN ROSVALL, IceLab, Department of Physics, Umeå University, Umeå, Sweden

Real-world networks have a complex topology comprising many elements often structured into communities. Revealing these communities helps researchers uncover the organizational and functional structure of the system that the network represents. However, detecting community structures in complex networks requires selecting a community detection method among a multitude of alternatives with different network representations, community interpretations, and underlying mechanisms. This tutorial focuses on a popular community detection method called the map equation and its search algorithm Infomap. The map equation framework for community detection describes communities by analyzing dynamic processes on the network. Thanks to its flexibility, the map equation provides extensions that can incorporate various assumptions about network structure and dynamics. To help decide if the map equation is a suitable community detection method for a given complex system and problem at hand—and which variant to choose—we review the map equation's theoretical framework and guide users in applying the map equation to various research problems.

Jelena Smiljanić and Christopher Blöcker contributed equally to this research.

J. S., D. E., and M. R. were supported by the Swedish Research Council, Grant No. 2016-00796. C. B. was supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation, the German Federal Ministry of Education and Research, Grant No. 100582863 (TissueNet), and the Swiss National Science Foundation, Grant No. 176938. A. H. and M. N. were supported by the Swedish Foundation for Strategic Research, Grant No. SB16-0089.

Authors' Contact Information: Jelena Smiljanić (corresponding author), IceLab, Department of Physics, Umeå University, Umeå, Sweden; e-mail: jelena.smiljanic@umu.se; Christopher Blöcker, IceLab, Department of Physics, Umeå University, Umeå, Sweden; e-mail: christopher.blocker@umu.se; Anton Holmgren, IceLab, Department of Physics, Umeå University, Umeå, Sweden; e-mail: anton.holmgren@umu.se; Daniel Edler, IceLab, Department of Physics, Umeå University, Umeå, Sweden; e-mail: daniel.edler@umu.se; Magnus Neuman, IceLab, Department of Physics, Umeå University, Umeå, Sweden; e-mail: magnus.neuman@umu.se; Martin Rosvall, IceLab, Department of Physics, Umeå University, Umeå, Sweden; e-mail: martin.rosvall@umu.se.

ACM 0360-0300/2026/02-ART183
https://doi.org/10.1145/3779648

CCS Concepts: • **General and reference → Surveys and overviews**; • **Mathematics of computing → Graph algorithms**; • **Information systems → Clustering and classification;**

Additional Key Words and Phrases: Networks, community detection, information theory, the map equation

ACM Reference Format:
Jelena Smiljanić, Christopher Blöcker, Anton Holmgren, Daniel Edler, Magnus Neuman, and Martin Rosvall. 2026. Community Detection with the Map Equation and Infomap: Theory and Applications. *ACM Comput. Surv.* 58, 7, Article 183 (February 2026), 34 pages. https://doi.org/10.1145/3779648

1 Introduction

Networks help researchers analyze complex systems by representing their intricate interactions. To simplify networks with numerous nodes and links and uncover their essential structure and dynamics, researchers have developed an array of methods to detect communities, also called modules [30, 32]. These methods vary in goals, assumptions, and interpretations, ranging from generative models representing network formation processes to flow-based approaches capturing processes on the network [74]. When choosing a method for a specific problem, users must consider their objectives, the system under study, and the available data [61]. The multitude of methods and settings can be daunting. Users often struggle to select the most suitable approach and configuration, leading to suboptimal results.

Two questions stand out when describing network communities: How did the network form? How does the network's structure affect processes on the network? While these questions are often intertwined, and community detection methods handle assumptions about the network's formation, structure, and dynamic processes differently, we can discern two major approaches. Different generative methods address the first question, such as community embedding models [80], community-affiliation graph models [84], and stochastic block models with Bayesian inference techniques [63]. These methods assume that a latent community structure determines the link distribution. Finding an optimal partition representing the underlying structure involves applying statistical inference to fit a generative model to the network data. Addressing the second question requires shifting focus to the processes on the network that uncover how nodes share similar dynamical roles.

In this review, network flows refer to dynamic processes on networks, such as the spread of information, activity, or contagion, which can often be modeled as random walks. This usage differs from the traditional meaning of network flow in computer science and operations research, where it typically refers to capacity-constrained optimization problems such as the max-flow min-cut problem [29]. Network flows, understood as dynamic processes on networks, are vital for our understanding of system-wide behavior in complex systems. They transform networks from combinatorial constructs of nodes and links into integrated representations of complex systems where distant parts can influence each other. Specifically, network flows capture the interconnected dynamics of complex systems, such as when element A interacts with element B and element B interacts with element C, elements A and C indirectly affect each other.

Sometimes, these flows are observable, such as passenger flows between airports in air traffic networks. Other times, we must model the flows. For example, when we want to comprehend genetic pathways but have access only to the gene network, we can derive flows from the interactions between pairs of genes. Since network structure constrains processes on it, studying modeled flows can also reveal meaningful structures in networks without explicit flows.

Flow-based modules coarse-grain the dynamics taking place on the network. Groups of nodes where the network flows stay relatively long provide an intuitive notion of flow-based modules. In

air traffic networks, passengers traveling remain within modules comprising sets of airports that contain numerous travel routes. In gene networks with unknown genetic pathways, functional modules trap modeled network flows representing the biological processes. In networks without explicit network flows, including synthetic benchmark networks with planted community structure of a generative model, flow-based methods have nevertheless proven effective in identifying these topological communities [1, 51, 86]. Because studying flow-based communities provides essential insights about the systems they represent, reliably identifying them is critical.

While there are several flow-based community detection methods [16, 48, 65, 75], we focus on the map equation and its search algorithm Infomap [21, 69]. The map equation captures modular regularities by compressing the description of flows, capitalizing on the minimum description length principle [66]. Applied to modular compression of network flows, the minimum description length principle states that finding the partition that enables the greatest compression of the network flows is equivalent to identifying the modules that best capture the regularities of those flows. Given a partition, the map equation calculates the lower bound for the per-step description length of a random walk on the given network. Thanks to the map equation's foundation in information theory and stochastic processes, it has proven efficient and accurate across various disciplines [1, 51, 86].

Over the years, the map equation has been extended in many directions, enabling researchers to analyze increasingly complex data across diverse applications. While powerful, this diversity creates a challenge: users struggle to navigate the variants and select the most suitable approach for their specific needs. Existing map equation tutorials focus on specific applications [11, 18], while textbooks and surveys offer high-level descriptions that lack the technical depth needed for practical application [3, 32]. This review bridges that gap. We synthesize the map equation's theoretical foundations with practical guidance, helping researchers choose and apply the appropriate variant for their community detection challenges. Throughout the review, we provide companion Jupyter notebooks with code examples related to the respective section, collected in a GitHub repository at https://github.com/mapequation/infomap-tutorial-notebooks and marked with ↗ in the text. We use schematic networks for clarity and include a few real-world examples when they offer additional insights.

2 Mapping Network Flows

Mapping network flows with the map equation framework comprises three steps: network representation, flow modeling, and flow mapping (Figure 1). Each step is adaptable and can be customized to effectively identify flow-based communities in a wide range of scenarios.

2.1 Network Representation

Complex systems comprise numerous diverse components that interact in intricate ways, making it impossible to comprehend the system's functioning through mere observation. Network representations seek to schematize the increasingly accessible relational data by abstracting away all but the essential: nodes and links in their most basic form. The effectiveness of a network representation relies on its ability to explain the complex system through subsequent analysis, including flow-based community detection.

The map equation framework can use a variety of network representations to reliably identify flow-based communities in complex systems (Figure 1). In the simplest case, we construct a network with binary links that indicate whether two elements interact. With more information about interactions, we can consider richer network representations: Directed and weighted networks to characterize the orientation and strength of interactions. Multilayer or multiplex networks when the problem requires distinguishing different interaction types or interactions that change over time. For capturing multi-body interactions, hypergraphs offer a suitable representation.

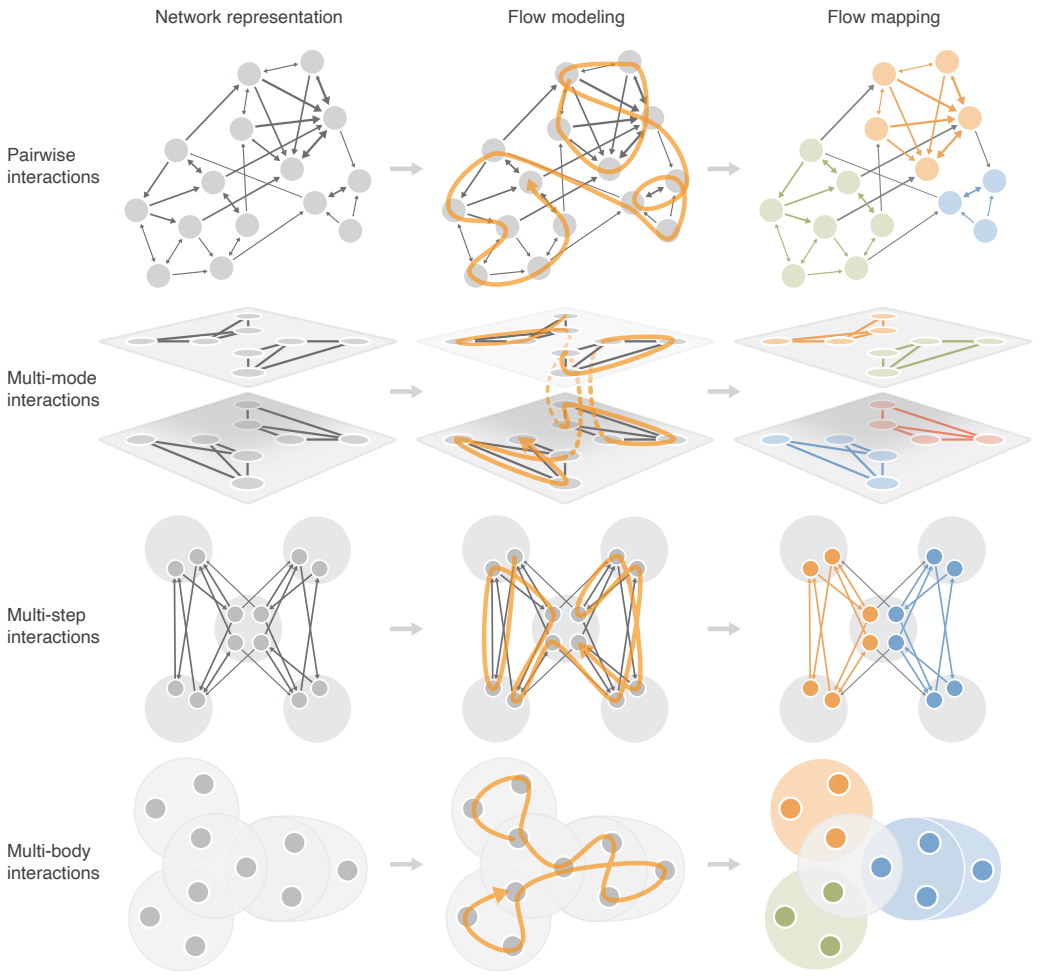

Fig. 1. **Modeling and mapping flow with the map equation framework.** Given complex system data, the researcher first selects an appropriate network representation (left column) based on the type of interactions: *Pairwise interactions* can be represented with weighted and directed networks, where link strength and direction capture interaction frequency and orientation. *Multi-mode interactions* call for multilayer networks, where nodes are replicated across layers representing different times, contexts, or modes. *Multi-step interactions* are captured with memory networks, where physical nodes (large circles) are associated with state nodes (smaller circles) that retain information about interaction sequences. *Multi-body interactions* among more than two nodes are naturally represented by hypergraphs, where hyperedges connect multiple nodes simultaneously. Next, a random walk model approximates real-world flow (middle column). Finally, minimizing the map equation reveals flow modules where a random walker remains for extended periods (right column). Because network flows reflect the systems' function, flow modules reveal the systems' functional components.

The research question guides how to best model the characteristics of a real-world complex system. Can a binary network capture the observed data sufficiently well, or is a weighted directed network required? Because adding weights and directions improves the model accuracy at a minimal computational cost in the map equation framework, when available, we incorporate them

with few exceptions. Will capturing salient temporal features require a multi-step or multi-mode representation? Are multi-body interactions indispensable for explaining system dynamics? These types of questions often require testing various network representations, comparing how they influence the flow model and the communities.

Because any network representation schematizes the raw interaction data, selecting the most suitable network representation involves a delicate balance between simplicity and accuracy: More intricate network representations prioritize accuracy at the cost of sacrificing conceptual simplicity and computational efficiency.

2.2 Flow Modeling

Unless the network flows are explicit, we build a flow model on top of the network representation (Figure 1). Modeling network flows as Markovian diffusion processes with random walks uses all the information in the network representation with minimal assumptions and can efficiently reveal flow-based communities. Higher-order generalizations of Markov processes enable incorporating the effects of multi-mode, multi-step, or multi-body interactions.

We first consider pairwise interactions and memoryless diffusion dynamics of a random walk on a network $G = (V, E)$ with nodes V, edges E, and $N = |V|$. In this setting, the network constrains the random walker's trajectory. The transition probability p_{uv} that a random walker at node u visits node v in the next step is proportional to the link weight w_{uv} between u and v,

$$p_{uv} = \frac{w_{uv}}{\sum_v w_{uv}}, \tag{1}$$

and the stationary node visit rates are given by the recursive system of equations

$$p_v = \sum_u p_u p_{uv}. \tag{2}$$

In undirected networks, the stationary distribution is proportional to node strength s,

$$p_v = \frac{s_v}{\sum_u s_u}, \tag{3}$$

where $s_v = \sum_u w_{uv} = \sum_u w_{vu}$ is node v's strength. That is, the probability that the random walker visits a node is high if the node has high strength. We refer to random walks to explain the principles behind the map equation, but in practice, we do not need to simulate random walks.

However, directed networks can have dangling nodes with no outgoing links or unreachable regions with no incoming links, so the node visit rates depend on where the random walker starts. In directed networks, random walks are ergodic so that Equation (2) converges to a unique solution only if the network is strongly connected and the random walk is aperiodic, according to the Perron–Frobenius theorem. Empirical networks often do not have these properties. To overcome this issue and ensure a stationary solution in directed networks, we use teleportation and allow the random walker to teleport to a random node independent of the current node at a low rate τ. Now, the stationary node visit rates can be obtained by solving equations

$$p_v^* = (1 - \tau) \sum_u p_u^* p_{uv} + \tau \varepsilon_v \tag{4}$$

with the power iteration method [36]. In Equation (4), the parameter ε_v represents the probability that the random walker teleports to the node v. This probability is uniform in the standard teleportation scheme, that is, $\varepsilon_v = \frac{1}{N}$ for all v [34].

The drawback of this teleportation strategy is that the stationary solution p_v^* depends on the choice of τ. We can apply unrecorded link teleportation, so-called smart teleportation [49], for more

robust results. Instead of assuming a constant probability ε_v where a random walker teleports to any node irrespective of the network topology, link teleportation selects a link as a teleportation target at random and proportional to its weight. This leads to a teleportation parameter ε_v proportional to a node's incoming strength

$$\varepsilon_v = \frac{s_v^{\text{in}}}{\sum_u s_u^{\text{in}}}, \tag{5}$$

where $s_v^{\text{in}} = \sum_u w_{uv}$ denotes the in-strength of node v. Link teleportation reduces the dependence on the parameter τ, however, encoding teleportation steps would affect the detected community structure as this corresponds to introducing artificial links between source and target nodes. Therefore, we do not record teleportation and only record steps along existing links. This amounts to performing an extra step without teleportation on a stationary solution for recorded teleportation

$$p_v^{\text{unrec}} = \sum_u p_u^* p_{uv}. \tag{6}$$

For any given network representation, all these flow models are unbiased such that a random walker follows links proportional to their weights. Unbiased random walks are default flow models in the map equation framework. Some applications may benefit from a more complex flow model. For example, a biased flow model where the random walker is guided by node attributes, preferring to visit nodes with similar attributes, can more accurately model real flow statistics with desirable effects on the flow mapping.

2.3 Flow Mapping

In modular networks, nodes connect more strongly within communities than between them. As the top row of Figure 1 illustrates, random walkers tend to stay inside communities for extended periods before exiting.

To capture these modular patterns, we capitalize on the correspondence between detecting regularities in data and compressing those data, formalized by the minimum description length principle [66]. Compression algorithms aim to represent data compactly by exploiting their regularities. Conversely, we aim to identify regularities in complex systems and measure our success by the compression they enable. The minimum description length principle applied to modular network flows formalizes this duality: the partition that best explains a network's modular flows is the one that most effectively compresses a description of the random walk (Figure 2).

To identify flow modules in a network using the minimum description length principle, we imagine a communication system using efficient codes to transmit the random walk's trajectory. As the random walker moves from node to node, it generates a sequence of node visits. If the network has modular regularities, so will the sequence. By identifying the structure that generates the sequence, we can describe the sequence more compactly with an efficient code [77]. For example, we can exploit frequently visited nodes within modules, much like text compression algorithms exploit repeated patterns. When the code structure mirrors the network's modular organization, the random walk description compresses more effectively.

Applying the minimum description length principle involves finding the model that can explain the data with the shortest possible code, trading off between model complexity and model fit. A schematic network with four modules illustrates (Figure 3): Placing all nodes in a single module minimizes the model complexity with a costly description of the random walker's position among all nodes in the network. The model maximally underfits the data. A two-module solution increases the model complexity, requiring specifying movements also between modules. But communicating the random walker's position given the module information is cheaper with fewer nodes to discern. The shortest possible description length decreases with three and four modules because the cost for

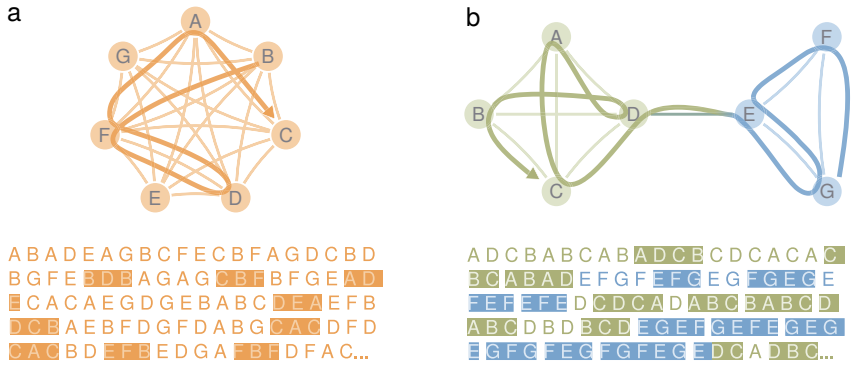

Fig. 2. **Compressing random walk descriptions in modular networks.** (**a**) In a fully connected network, the random walker visits all nodes uniformly, producing node-visit sequences shown at the bottom. Because repetitions occur randomly, modular compression offers no benefits. (**b**) In a modular network, the walker tends to stay within communities, and the corresponding sequences at the bottom reveal repeated patterns that enable modular compression. Recurring subsequences of length greater than two are highlighted with colored backgrounds.

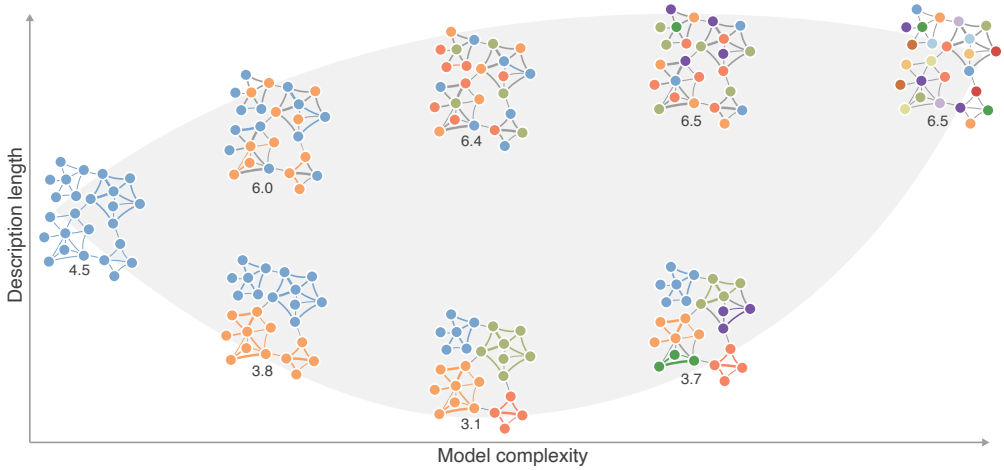

Fig. 3. **Schematic solution landscape of varying model complexity.** The number of modules in each network partition defines its model complexity, increasing from left to right. Colors indicate module assignments, and the numbers below solutions approximate the description lengths. The solution with the lowest description length balances model complexity and modular regularities.

model complexity increases slower than the description of the random walker's position given the module information. Increasing the module complexity beyond four modules leads to overfitting and a higher description length in this example. The shortest cost for specifying the position in smaller modules cannot compensate for more complex models. The model with one module for each node has the highest model complexity, maximally overfitting the data.

3 The Map Equation

To employ the minimum description length principle, we conceptually encode the random walker's trajectory with codewords assigned to nodes. The code's average per-step description length—

we call it *codelength* for short—measures how well the code exploits regularities in the walker's trajectory. In practice, we consider not the code itself but the modular compression it enables: the codelength. Importantly, we can analytically quantify how well different partitions compress expected trajectories without simulating random walks or explicitly assigning codewords.

Grouping nodes into modules enables reusing codewords across modules with one designated codebook per module for a shorter overall codelength. However, this requires introducing an index-level codebook and module-specific module-exit codewords for describing transitions between modules, increasing the overall codelength. Finding modules that describe the random walk's regularities well to minimize the codelength means balancing between choosing small modules for efficiently describing intra-module steps and choosing modules such that there are only few links between modules for minimizing the rate at which the random walker takes inter-module steps.

Per Shannon's source coding theorem [77], the codelength's lower limit when not partitioning nodes into modules is the entropy over the nodes' stationary visit rates. Applied to a network's modular structure, the codelength's lower limit is the sum of the module and index-level codebooks' entropies, weighted by the rate at which they are used.

3.1 The Two-level Map Equation

For a partition M of the nodes $u \in V$ into modules $\mathsf{m} \in \mathsf{M}$, we construct $|\mathsf{M}|$ module codebooks. This formulation assumes non-overlapping communities, where each node belongs to exactly one module. Each node u has a module-dependent codeword, meaning that the same codeword can be reused for nodes in other modules. For a uniquely decodable code, an additional index codebook encodes transitions into modules. The index codewords derive from the frequencies of module entries. The probability $q_{\mathsf{m}\curvearrowleft}$ that a random walker enters module m is

$$q_{\mathsf{m}\curvearrowleft} = \sum_{u \notin \mathsf{m}, v \in \mathsf{m}} p_u p_{uv}. \tag{7}$$

Besides codewords for communicating which node a random walker visits, each module codebook requires a designated exit codeword for communicating when a random walker leaves the module. The probability $q_{\mathsf{m}\curvearrowright}$ of leaving module m is

$$q_{\mathsf{m}\curvearrowright} = \sum_{u \in \mathsf{m}, v \notin \mathsf{m}} p_u p_{uv}. \tag{8}$$

To define the map equation, we apply Shannon's source coding theorem to the index codebook and each module codebook. The codelength for describing movements within module m is

$$H(P_{\mathsf{m}}) = -\frac{q_{\mathsf{m}\curvearrowright}}{p_{\mathsf{m}}^{\circlearrowleft}} \log_2 \frac{q_{\mathsf{m}\curvearrowright}}{p_{\mathsf{m}}^{\circlearrowleft}} - \sum_{u \in \mathsf{m}} \frac{p_u}{p_{\mathsf{m}}^{\circlearrowleft}} \log_2 \frac{p_u}{p_{\mathsf{m}}^{\circlearrowleft}}, \tag{9}$$

where $p_{\mathsf{m}}^{\circlearrowleft} = q_{\mathsf{m}\curvearrowright} + \sum_{u \in \mathsf{m}} p_u$ is the total use rate of module m's codebook. Similarly, for the index codebook, the codelength for describing random walker transitions between modules is

$$H(Q) = -\sum_{\mathsf{m} \in \mathsf{M}} \frac{q_{\mathsf{m}\curvearrowleft}}{q_{\curvearrowleft}} \log_2 \frac{q_{\mathsf{m}\curvearrowleft}}{q_{\curvearrowleft}}, \tag{10}$$

where $q_{\curvearrowleft} = \sum_{\mathsf{m} \in \mathsf{M}} q_{\mathsf{m}\curvearrowleft}$.

The sum of codelengths for all codebooks weighted by their use rate gives the map equation

$$L(\mathsf{M}) = q_{\curvearrowleft} H(Q) + \sum_{\mathsf{m} \in \mathsf{M}} p_{\mathsf{m}}^{\circlearrowleft} H(P_{\mathsf{m}}). \tag{11}$$

With an increasing number of modules, the codelength of the module codebooks decreases while the codelength of the index codebook increases. The network partition that best captures the modular network flows maximally compresses the description length defined by the map equation—minimizing the map equation over all possible partitions gives the optimal partition.

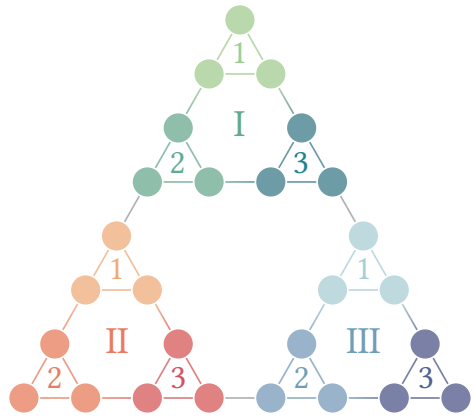

Fig. 4. **Schematic triangle network with a multilevel partition.** Each small triangle (1, 2, 3) is a sub-module, organized in groups of three triangles in each top-level supermodule (I, II, III).

3.2 The Multilevel Map Equation

Complex systems often have a hierarchical—*multilevel*—organization consisting of nested modules at different levels (Figure 4). We generalize the map equation to describe multilevel structures by applying it to each module $\mathsf{m} \in \mathsf{M}$ recursively [18, 71],

$$L(\mathsf{M}) = q_{\curvearrowleft} H(Q) + \sum_{\mathsf{m}\in\mathsf{M}} L(\mathsf{m}). \tag{12}$$

Each module m contains either (i) a set of submodules or (ii) a set of nodes:

i. For a module m with submodules $\mathsf{m}' \in \mathsf{m}$, the contribution to the overall codelength is

$$L\left(\mathsf{m}\right) = q_{\mathsf{m}}^{\circlearrowright} H\left(Q_{\mathsf{m}}\right) + \sum_{\mathsf{m}'\in\mathsf{m}} L\left(\mathsf{m}'\right), \tag{13}$$

that is, we treat each module m with submodules the same way as the top-level module M, with one addition: We need to consider module exits from m, thus defining its codebook usage rate

$$q_{\mathsf{m}}^{\circlearrowright} = q_{\mathsf{m}\curvearrowright} + \sum_{\mathsf{m}'\in\mathsf{m}} q_{\mathsf{m}'\curvearrowleft} \tag{14}$$

and

$$H\left(Q_{\mathsf{m}}\right) = -\frac{q_{\mathsf{m}\curvearrowright}}{q_{\mathsf{m}}^{\circlearrowright}} \log_2 \frac{q_{\mathsf{m}\curvearrowright}}{q_{\mathsf{m}}^{\circlearrowright}} - \sum_{\mathsf{m}'\in\mathsf{m}} \frac{q_{\mathsf{m}'\curvearrowleft}}{q_{\mathsf{m}}^{\circlearrowright}} \log_2 \frac{q_{\mathsf{m}'\curvearrowleft}}{q_{\mathsf{m}}^{\circlearrowright}}. \tag{15}$$

ii. The codelength contribution of a module m that contains nodes but no further submodules is

$$L\left(\mathsf{m}\right) = p_{\mathsf{m}}^{\circlearrowright} H\left(P_{\mathsf{m}}\right), \tag{16}$$

where

$$p_{\mathsf{m}}^{\circlearrowright} = q_{\mathsf{m}\curvearrowright} + \sum_{u\in\mathsf{m}} p_u \tag{17}$$

and

$$H(P_{\mathsf{m}}) = -\frac{q_{\mathsf{m}\curvearrowright}}{p_{\mathsf{m}}^{\circlearrowright}} \log_2 \frac{q_{\mathsf{m}\curvearrowright}}{p_{\mathsf{m}}^{\circlearrowright}} - \sum_{u\in\mathsf{m}} \frac{p_u}{p_{\mathsf{m}}^{\circlearrowright}} \log_2 \frac{p_u}{p_{\mathsf{m}}^{\circlearrowright}}. \tag{18}$$

3.3 Challenges and Remedies

Resolution limit. Finding modules in large networks can be affected by a resolution limit that prevents detecting small modules when large modules are present. Small but functionally meaningful modules may be merged into larger ones.

For the map equation, there is an analytical estimate that relates the number of links between modules to the map equation's resolution limit,

$$\frac{4^{l_{\mathsf{m}}}}{l_{\mathsf{m}}+1} \lesssim C, \tag{19}$$

where l_{m} is the number of internal links in module m, and C is the cut size, that is, the number of links between modules [46]. However, compared to other widely used community detection methods, such as Modularity, whose smallest detectable module scales as $O(\sqrt{E})$ with the total number of links in the network [31], and the stochastic block model, which scales as $O(\sqrt{N})$ with the total number of nodes [62], the map equation is less affected by the resolution limit.

Field-of-view limitations. Due to the resolution limit, which can prevent detecting modules below an effective size, methods can have an upper limit in the effective size they can detect, a so-called field-of-view limit. While the map equation is less susceptible to the resolution limit than other community detection methods, the field-of-view limit can affect its performance in some networks.

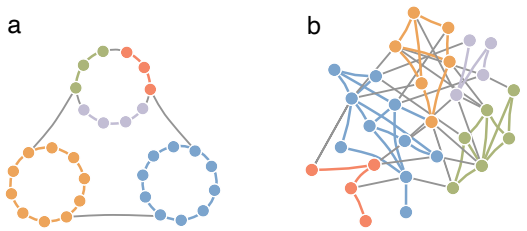

Fig. 5. **Field-of-view limitations.** Over-partitioning can occur in networks with (a) constrained structure, or (b) random structure. Node colors show module assignments.

The map equation's flow-based approach implicitly assumes that modules are assortative structures where nodes with similar functions are densely connected. However, this assumption may not be valid in networks with constrained structures. For example, geographically embedded networks such as power grids or transportation networks typically contain sparse regions. The map equation tends to over-partition these regions into smaller communities [75, 76]. We illustrate a realization of this problem in Figure 5(a).

Over-partitioning can also occur in random networks created without an explicit modular structure. The map equation aims to describe the network's structure concisely but without assuming any generative process. Consequently, the map equation identifies modular structure in random networks whose density is below a certain threshold, simply because such networks contain subgroups of nodes with low external link density [51] (Figure 5(b)).

Markov time scaling. By integrating Markov time scaling with the map equation to adjust its resolution, we can overcome the field-of-view limitations [47]. The standard map equation encodes the random walker's position in the network after every transition; thus, the default Markov time is $t = 1$. However, we can generalize the map equation to Markov times other than one by scaling the transition rates in Equation (1) with the Markov time t

$$p_{uv}(t) = tp_{uv}, \tag{20}$$

and if $t < 1$, we additionally account for

$$p_{uu}(t) = (1-t) + tp_{uu}. \tag{21}$$

This scaling approximates continuous-time transition probabilities between nodes u and v, which grow linearly with time. If we let the walker take t steps before encoding its position, it transitions

between nodes approximately t times more than if it only took one step. While the stationary distribution of node visits remains unchanged, the transition rates between nodes increase in proportion to t. For $t < 1$, the random walker visits the same node several times because it moves slower, resulting in smaller modules. For $t > 1$, the walker moves faster and can take more than one step before we encode, such that not every node along the trajectory is encoded, favoring larger modules.

Variable Markov time scaling. Variable Markov time [22] relaxes the constraints of a global Markov time parameter, choosing between detecting large-scale structures above the field-of-view limit or highlighting small dense structures. The basic idea is to adjust the Markov time dynamically based on the random walker's current neighborhood in the network, allowing it to move faster in sparse areas and slower in dense areas, reducing the effects of both the field-of-view limit and the resolution limit simultaneously. Adjusting the random walker's Markov time dynamically enables capturing a broader range of modular flow patterns within a single map.

To scale the Markov time dynamically, we adjust the flow between nodes with a node-local Markov time t_u that is inversely proportional to the flow of node u. As a baseline, we use the densest part of the network and keep the minimum Markov time at 1 to avoid the resolution limit. We define the variable Markov time t_u for node u as the ratio between the maximum flow over all nodes, $p_{\max}$, and the local flow p_u,

$$t_u = \frac{p_{\max}\, k_{\text{tot}}}{\min(1, p_u k_{\text{tot}})}, \tag{22}$$

where $k_{\text{tot}} = \sum_u k_u$ is the total node degree over all nodes in the network. However, in real-world networks where most nodes typically have few links and a few nodes have many links, this definition of variable Markov time is sensitive to the degree of the strongest node. We can address this issue by taking the logarithm with base 2 of $p_{\max}$ and p_u instead to avoid small Markov times, resulting in a variable Markov time where the random walker moves with a constant entropy rate

$$t_u = \frac{\log_2 (p_{\max}\, k_{\text{tot}})}{\min\left(1, \log_2 (p_u\, k_{\text{tot}})\right)}. \tag{23}$$

For directed links, we scale the flow with the Markov time of the source nodes. To keep the minimum Markov time at 1 for undirected links, we scale the flow with the minimum Markov time $t_{uv} = \min(t_u, t_v)$ between nodes u and v. Typically, networks with less skewed degree distribution can benefit from using Equation (22) to avoid the field-of-view limit.

4 Infomap

Infomap is a greedy stochastic search algorithm designed to minimize the map equation and detect two-level and multilevel flow communities in networks [11, 71]. Since community detection is an NP-hard combinatorial optimization problem, Infomap cannot guarantee to find the map equation's global minimum; it uses iterative and recursive optimization heuristics to avoid local minima and is known to achieve good results in practice [1, 51, 86]. The Infomap search algorithm is inspired by the Louvain algorithm for modularity maximization [10] but uses additional fine-tuning and coarse-tuning steps, similar to how the Leiden algorithm later refined Louvain [82]. When expressed in differentiable tensor form with soft cluster assignments, the map equation can also be optimized with graph neural networks [9]. Infomap is available as an open-source software package [21].

Infomap runs in two phases: the first phase creates a two-level partition, and the second phase creates a multilevel partition. Depending on the use case, Infomap can stop either after the two-level phase and output a two-level partition or after the multilevel phase for a multilevel partition if it improves the two-level solution.

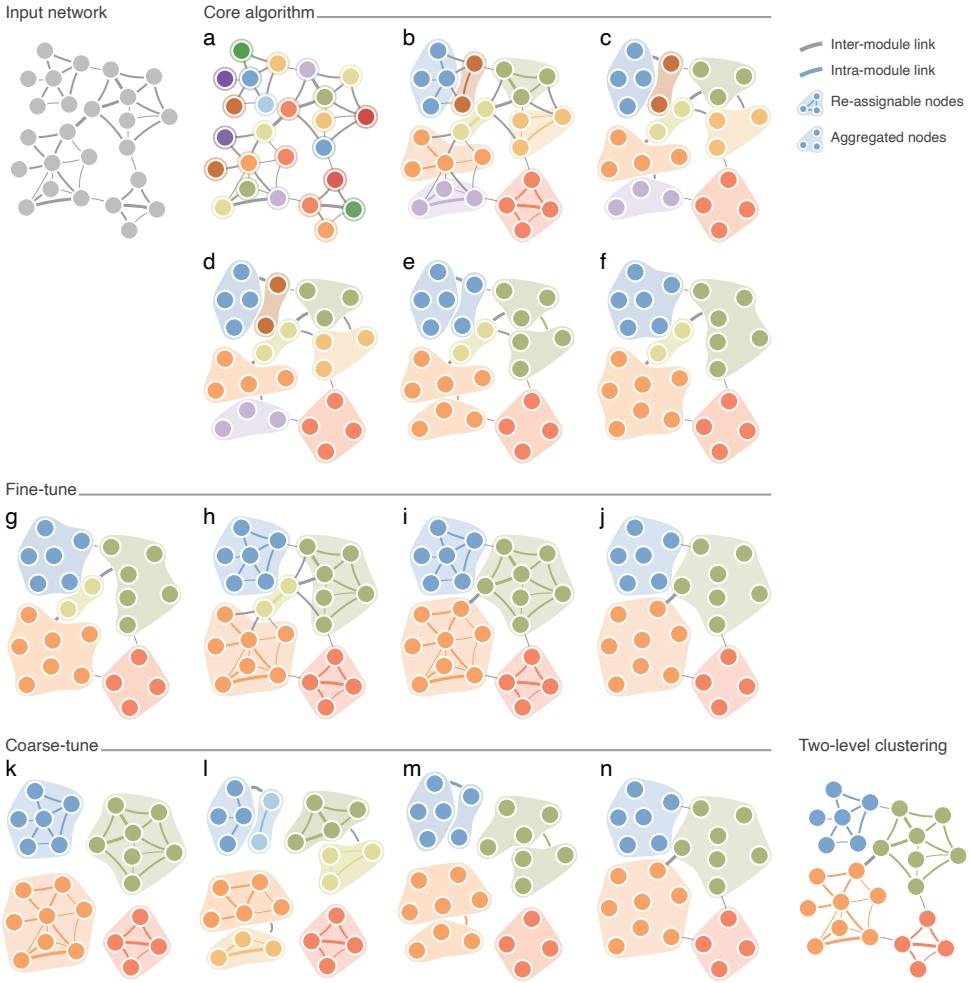

Fig. 6. **Schematic illustration of Infomap's search algorithm for two-level partitions.** Infomap uses a greedy stochastic search algorithm to optimize the map equation over modular regularities. The algorithm combines multiple strategies to avoid local minima. It begins by placing each node in its own module (a), then iteratively moves nodes to neighboring or new modules if such moves reduce the codelength (b–c). Next, it moves the identified modules together as a unit and repeats the same logic at a higher level (d–f). This recursive grouping repeats as long as the total codelength decreases beyond a minimal threshold. To refine module assignments at multiple scales, Infomap alternates between fine-tuning (g–j) and coarse-tuning (k–n). During fine-tuning, it revisits node-level assignments (b–c) starting from the current modular structure, again moving individual nodes between modules to reduce the codelength (g–j). During coarse-tuning, Infomap first partitions each module independently to identify sub-modular groups (l), reassigns these groups to the global modular structure (m), and moves them between modules using the higher-level logic (d–f), continuing as long as the codelength decreases beyond a minimal threshold (n).

4.1 The Two-level Phase

The two-level phase consists of two stages:

In stage 1, first, Infomap assigns each node to its own module, creating a trivial partition with high codelength as a starting point (Figure 6(a)). Such a partition of singletons places all links

between modules—assuming the network has no self-loops—and forces the random walker to switch modules with every step, inefficiently compressing the random walker's trajectory. Second, Infomap updates the partition iteratively and considers moving nodes in random order to either of their neighboring modules or a new singleton module (Figure 6(b)). To avoid re-checking nodes with stable module assignments, Infomap initially marks all nodes as candidates for moving. Then, as long as there are candidates left, Infomap randomly chooses a candidate and tries to move it to reduce the codelength by some adjustable minimum amount. If there are several possible moves for a node, Infomap chooses the one that reduces the codelength the most. When Infomap moves a node, it marks that node and all its neighbors as candidates for moving. A node that has no improving move is unmarked. When no candidate nodes are left, Infomap stops moving individual nodes (Figure 6(c)). Third, Infomap repeats the second step, now moving the identified modules: nodes that belong to the same module are moved into or out of other modules together as a group. This step repeats with larger and larger modules as long as the codelength decreases (Figure 6(d)–(f)). We call stage 1 Infomap's *core algorithm*.

In stage 2, Infomap tunes the partition to avoid local minima by alternating between fine-tuning and coarse-tuning steps. Fine-tuning and coarse-tuning follow the same principle and use the core algorithm, but they consider different objects to move. For fine-tuning, Infomap uses the core algorithm to move individual nodes to improve the codelength (Figure 6(g)–(j)). For coarse-tuning, Infomap partitions all modules individually into sub-modules using the core algorithm and then moves these sub-modules to reduce the codelength (Figure 6(k)–(n)). Infomap fine- and coarse-tunes at least once and stops when it cannot improve the codelength by some minimum amount: either by an absolute value or a relative value determined from the codelength of the one-level partition, both of which can be adjusted.

4.2 The Multilevel Phase

The multilevel phase aims to reduce the codelength by adding further index levels to a two-level partition. It contains two stages.

In stage 1, Infomap compresses inter-module transitions by first aggregating the network at the module level. This creates a network where nodes represent the previous modules, and inter-module links are merged. Second, Infomap uses the two-level algorithm to partition the aggregated network. The resulting two-level partition comprises a three-level partition when interpreted in the context of the network before aggregation. Infomap repeats stage 1 as long as aggregating and partitioning the network and adding one more index level per iteration yields a non-trivial solution.

In stage 2, Infomap partitions the modules at the highest level of the multi-level partition recursively. Since the lower levels in the partition were designed from a perspective that did not consider the higher levels, they may be suboptimal and suffer from resolution issues. Moreover, the resulting partition is balanced, and all modules have the same depth, which is generally not the case in real-world networks. To overcome this, Infomap keeps the modules at the highest level but forgets their submodules to re-partition each module independently, using the full algorithm, that is, the two-level phase followed by the multi-level phase.

4.3 Solution Landscape

Even though Infomap is designed to avoid converging to a local minimum, the non-convex properties of the map equation can be a problem in practice. Moreover, local minima can have a codelength close to the global minimum, which can be an inherent property of the studied system or be caused by noisy data or sensitivity to model parameters.

To study the map equation's solution landscape, we visualize the detected minima [12]. We run Infomap repeatedly and with different seeds to generate different partitions and cluster them to

identify the partition clusters in the map equation's solution landscape. We consider the solution landscape complete when new partitions fall within already identified partition clusters with a high probability. Reaching a complete landscape also answers how many times we need to run Infomap to find all relevant solutions. To measure distances between partitions, we use a Jaccard-based metric that allows using two-level or multilevel partitions. By varying the cluster distance threshold we can explore the solution landscape at different scales.

Figure 7 shows a two-dimensional embedding of the jazz network's [35] solution landscape, created with UMAP [55]. The jazz network's solution landscape has several local minima with codelengths close to the global minimum. If these alternative partitions differ in any aspect important to the researcher—compared to each other or the global minimum—they can reveal important aspects of the underlying data. In contrast, the solution landscape of the Zachary karate club network [85] has only one partition, that is, Infomap always converges to the same solution.

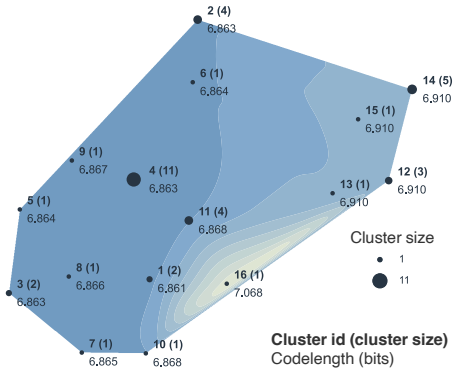

Fig. 7. **The solution landscape of the jazz network.** Visualization of clusters of partitions and the codelength of the best partition in each cluster. Cluster 1 has two partitions and the shortest codelength. Clusters 2–4 have a slightly higher codelength and comprise 17 partitions. Differences between partitions of similar quality can reveal important aspects of the underlying data. Color indicates the solution quality, with light green for long codelengths and dark blue for short codelengths. The UMAP embedding preserves local relationships; similar solutions appear close to each other in the solution landscape.

5 The Map Equation for Higher-order Networks

Higher-order network models can provide a more accurate representation of complex systems by capturing dynamics beyond pairwise interactions [5, 81]. These networks have found applications in various fields where conventional network models may fall short. For example, conventional networks may struggle to represent protein–protein interactions, where three or more proteins can form complexes. Similarly, they may not adequately capture the dynamics of social networks, where the information flow depends on its source.

To address these limitations, the higher-order map equation framework introduces so-called state nodes to represent higher-order dependencies and generalizes Markovian dynamics from first to higher order. State nodes can reflect various aspects of the system, such as different layers in multilayer networks or previously visited physical nodes in memory networks. Physical nodes correspond to regular nodes in first-order networks and represent the interacting entities.

We use the map equation to partition nodes into communities that minimize the description of random walker movements between state nodes. This generalization does not impact the index codebook but necessitates modifications to the module codebooks. When multiple state nodes of the same physical node are assigned to the same module, they share the same codeword to ensure they represent the same object. At the same time, state nodes of the same physical node assigned to different modules allow identifying overlapping modules.

We discuss various higher-order models focusing on memory networks, multilayer networks, temporal networks, and hypergraphs. While the map equation directly supports multilayer and memory networks, temporal networks and hypergraphs require additional modeling choices. We build on the memory and multilayer map equation and incorporate additional constraints into the random-walk model to capture the intricacies of temporal and multi-body interactions.

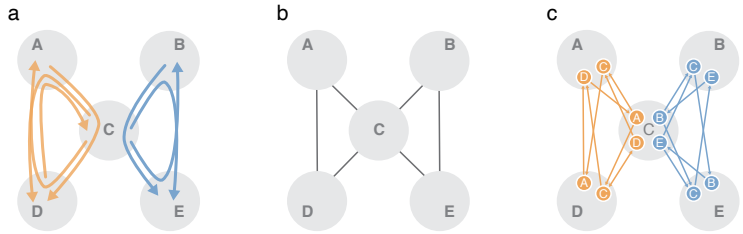

Fig. 8. **Two-step paths represented as a memory network.** Two-step paths (a) are realized as a first-order network in (b) and a second-order memory network using state nodes in (c). Each state node represents the memory of the previous step: The path A – C – D is represented by the state node A in physical node C connected to the state node C in D. With more pathway data, each physical node may comprise multiple state nodes. The memory network can be partitioned into overlapping modules by assigning the state nodes belonging to the same physical node to different modules.

5.1 Memory Networks

In a kth order memory network, the random walker's next step depends on the $k-1$ previously visited nodes

$$P_{uv} = P(v|u, u_{-1}, \ldots, u_{-k+1}), \tag{24}$$

where u is the current node, u_{-1} the previous node, and u_{-k} the node visited k steps ago. Modeling with memory networks enables representing flow phenomena such as high return probabilities. Memory networks have been used successfully in various domains, including modeling citation pathways through multidisciplinary journals and traffic flow through transit airports [64, 72].

We illustrate the approach using flows from two domains through a central node C (Figure 8(a)). Conventional first-order networks cannot model the flow dependency on their origin, resulting in undesired mixing (Figure 8(b)). In contrast, a second-order memory network overcomes this limitation and encodes the history of previously visited physical nodes in state nodes: Here, the state node A in physical node C represents the transition from A to C (Figure 8(c)). Using state nodes allows us to separate between flows that mix in physical node C and identify two overlapping modules.

Memory networks from first-order data. To obtain a stationary flow distribution for state nodes in a memory network, we require sequence data; ideally, empirical multi-step data. However, in cases where such data are unavailable, we can model higher-order data on top of first-order networks to reveal overlapping modules. This approach offers a valuable alternative for identifying patterns in complex systems with higher-order dependencies, even in the absence of empirical multi-step data.

For example, we can use the second-order random walk model known as *node2vec* [38] to create state nodes. In this model, the return parameter p controls the probability that flows return to where they came from, while the in-out parameter q controls the probability that flows stay in the vicinity or move further away from where they came from. By tuning p and q, flows can be directed to stay more local to where they came from, reinforcing triangles [54]. To avoid representing every possible second-order pathway as state nodes, we can derive a compact memory network by lumping state nodes based on minimum information loss [54].

5.2 Multilayer Networks

Real-world systems are often characterized by flows that can be stratified across different layers. For example, individuals may use different modes of transport to commute between cities or

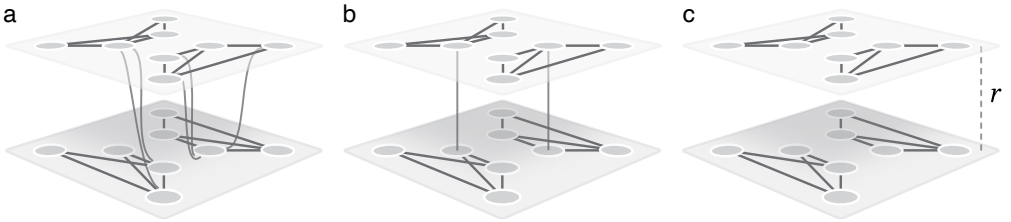

Fig. 9. **Multilayer networks.** Multilayer network models with (a) explicit multilayer links, (b) intra- and inter-layer links, and (c) intra-layer links and inter-layer relax rate.

communicate with each other using various messaging or social media platforms. To model such multilayer flows, we utilize multilayer networks where each physical node can exist in many layers with different link structures. A group of nodes with similar connectivity patterns across different layers should be assigned to the same community. However, if the connectivity patterns of a physical node vary across the layers, it should be assigned to multiple communities.

The map equation interprets a multilayer network with l layers as an lth-order memory network where state nodes assigned to a physical node correspond to different layers [15]. When connecting multilayer state nodes, we distinguish two types of links: intra-layer links, which connect nodes within the same layer, and inter-layer links, which connect nodes across different layers. Depending on the input data, there are three ways to define inter-layer links.

Explicit multilayer links. If the data provide explicit information about the link weight $w_{u^A v^B}$ between node u in layer A and node v in layer B (Figure 9(a)), we can directly obtain a stationary flow distribution for the state nodes. The transition probability between any two state nodes u^A and v^B is

$$p_{u^A v^B} = \frac{w_{u^A v^B}}{\sum_{v^B} w_{u^A v^B}}. \tag{25}$$

Intra- and inter-layer links. In some cases, the data lack explicit multilayer links between all pairs of state nodes. Instead, it may provide explicit links within layers, and inter-layer link weights $\upsilon_{u^A u^B}$, which describe the coupling between physical node u in layers A and B when $A \neq B$ (Figure 9(b)). In this model, the random walker moves between layers proportional to inter-layer link weight, but we do not encode this movement. The map equation expands the inter-layer links to multilayer links, and the transition probability is

$$p_{u^A v^B} = \frac{\upsilon_{u^A u^B}}{\sum_B \upsilon_{u^A u^B}} \frac{w_{u^B v^B}}{\sum_v w_{u^B v^B}}, \tag{26}$$

where $\upsilon_{u^A u^A} = \sum_v w_{u^A v^A}$.

Intra-layer links and inter-layer relax-rate. In most cases, empirical inter-layer links are not provided. To relax the layer constraints and allow the random walker to move without inter-layer links, the map equation introduces a relax rate parameter r (Figure 9(c)). In this model, the random walker follows intra-layer links with a probability $1-r$ and moves between layers with a probability r. With the Kronecker delta δ, the transition probabilities become

$$p_{u^A v^B} = (1-r)\,\delta_{AB} \frac{w_{u^A v^B}}{\sum_{v^B} w_{u^A v^B}} + r \frac{w_{u^A v^B}}{\sum_A \sum_{v^B} w_{u^A v^B}}. \tag{27}$$

5.3 Modeling Temporal Data

Memory and multilayer networks offer two alternatives to capture higher-order dependencies in temporal network data. Memory networks capture causal paths that standard networks wash

out [50]. For example, A can only causally influence C through B if A influences B before B influences C, but conventional first-order networks discard temporal ordering and distort essential flow dynamics critical for unveiling system functions (Figure 10). In contrast, state nodes in memory networks preserve temporal ordering.

When dealing with network snapshots across different time intervals rather than pathway data, we model the data with a temporal multilayer network. Such a model allows us to represent the network's evolution over time by organizing snapshots into distinct layers and enables analyzing, for example, temporal communities in ecological systems and fossil record data [28, 67]. In these systems, inter-layer links are typically absent; instead, a relaxation rate models random-walker transitions between layers. To control temporal ordering in multilayer networks explicitly, the map equation can apply layer constraints to restrict the random walker to jump only to layers within a specific temporal distance [2].

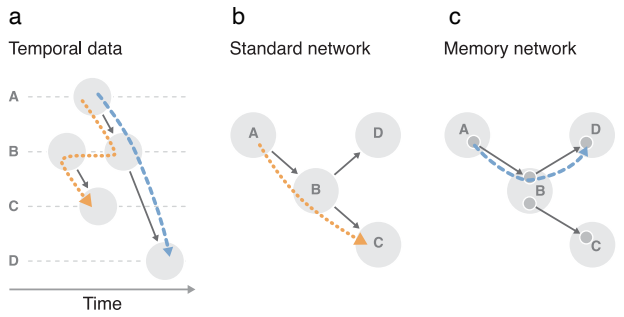

Fig. 10. **Standard network models destroy temporal information.** (a) Drawing interactions along a time axis shows that the orange random walker moves back in time, thus breaking causality. The blue random walker guided by state nodes obeys causality. (b) Conventional first-order network models allow the orange random-walker step because they do not model temporal dependencies. (c) Memory networks constrain temporal flows according to available data.

Furthermore, an adapted version of the map equation captures intermittent communities that emerge and dissolve repeatedly and independently from other communities in the network [2]. Because the multilayer map equation couples entire layers, it cannot accurately capture flow persistence within groups of nodes whose existence is time-dependent, making it difficult to identify intermittent communities. To address this scenario, we use node-level layer coupling: Node-level layer coupling estimates the coupling strength $v_{u^A u^B}$ between node u in layers A and B based on how similar u's connectivity patterns are in these two layers, quantified using the Jensen-Shannon divergence (JSD)

$$v_{u^A u^B} = 1 - JSD(p_{u^A}, p_{u^B}), \tag{28}$$

where $p_{u^A} = (p_{u^A v_1^A}, \ldots, p_{u^A v_N^A})$ represents the intra-layer probabilities that a random walker steps from node u in layer A to other nodes v in layer A.

5.4 Modeling Multi-body Interactions

We can identify modules in hypergraphs by representing random walks on hypergraphs as walks on conventional or multilayer networks. However, different systems and research questions necessitate different random-walk and hypergraph models. Hyperedges can have weights, and nodes can have hyperedge-dependent weights [14]. Random walks can be lazy, allowing multiple consecutive visits to the same node, or non-lazy, forcing it to move on [13]. Additionally, these random-walk models can be represented using various network types, such as bipartite, unipartite, or multilayer networks. What network representation to choose depends on the research question, as each has different advantages.

We model flows on hypergraphs with random walks, using hypergraphs with nodes V, hyperedges E with weights ω, and hyperedge-dependent node weights γ. Each hyperedge e has a weight $\omega(e)$.

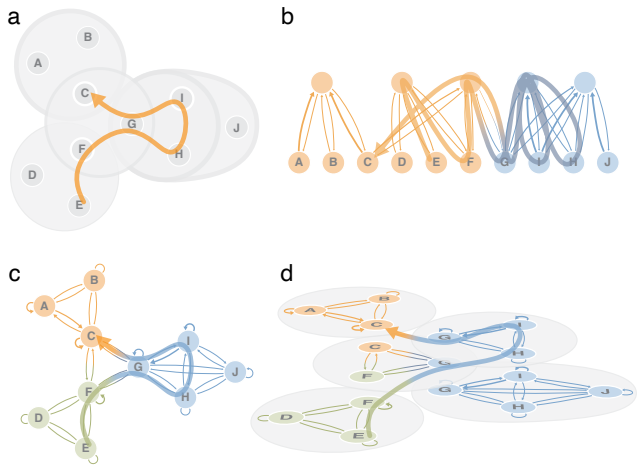

Fig. 11. **Schematic hypergraph represented with three types of networks.** (a) The schematic hypergraph with weighted hyperedges and hyperedge-dependent node weights. White circles labeled from A to J represent nodes, and large circles represent hyperedges incident to the nodes in each circle. Node and hyperedge borders indicate weight. A random walk depicted with an arrow on the schematic hypergraph represented on: (b) a bipartite network where the unlabelled nodes represent the hyperedges, (c) a unipartite network, and (d) a multilayer network with grey circles defining each layer. Node colors indicate optimized module assignments, links' thicknesses are proportional to the random walk's transition rates. The figure is adapted from Ref. [26], licensed under CC BY 4.0.

Each node u has a weight $\gamma_e(u)$ for each hyperedge e incident to u. We denote node u's total incident hyperedge weight $d(u) = \sum_{e \in E(u)} \omega(e)$ and hyperedge e's total node weight $\delta(e) = \sum_{u \in e} \gamma_e(u)$. With these weights, a lazy random walker moves from node u to v in three stages (Figure 11)[14]

(1) Picking hyperedge e among node u's hyperedges $E(u)$ with probability $\frac{\omega(e)}{d(u)}$.
(2) Picking one of hyperedge e's nodes v with probability $\frac{\gamma_e(v)}{\delta(e)}$.
(3) Moving to node v.

Variations include non-lazy walks, which never visit the same node twice in a row, with a modified second stage

(2b) Picking one of hyperedge e's nodes $v \neq u$ with probability $\frac{\gamma_e(v)}{\delta(e)-\gamma_e(u)}$.

Bipartite. Bipartite networks offer the most direct representation of the basic three-stage random-walk process. We represent the hyperedges with *hyperedge nodes*, and the three stages become a two-step walk between the nodes at the bottom and the hyperedge nodes at the top in Figure 11(b). First, a step from a node u to a hyperedge node e,

$$P_{ue} = \frac{\omega(e)}{d(u)}, \tag{29}$$

and then a step from the hyperedge node to a node v,

$$P_{ev} = \frac{\gamma_e(v)}{\delta(e)}. \tag{30}$$

For non-lazy walks, we let each incoming link to a hyperedge node connect to a state node with out-links to all of the hyperedge's nodes except the incoming link's source node.

Unipartite. To represent the random walk on a unipartite network, we project the three-stage random-walk process down to a one-step process between the nodes and describe it with the transition rate matrix

$$P_{uv} = \sum_{e \in E(u,v)} P_{ue} P_{ev} = \sum_{e \in E(u,v)} \frac{\omega(e)}{d(u)} \frac{\gamma_e(v)}{\delta(e)}, \tag{31}$$

where $E(u, v)$ is the set of hyperedges incident to both nodes u and v. Each hyperedge forms a fully connected group of nodes (Figure 11(c)). Unipartite networks for non-lazy walks have no self-links. The unipartite representation forms a weighted one-mode projection of the bipartite representation and requires more links with its fully connected groups of nodes.

Multilayer. To represent the random walk on a multilayer network, we project the three-stage random-walk process down to a one-step process on state nodes in separate layers. Each hyperedge e with weight $\omega(e)$ forms a layer A with weight $\omega(A)$. A state node u^A represents u in each layer $A \in E(u)$ that contains the node. All state nodes in the same layer form a fully connected set (Figure 11(d)). The transition rate between state node u^A in layer A and state node v^B in layer B is

$$P_{uv}^{AB} = \frac{\omega(B)}{d(u)} \frac{\gamma_B(v)}{\delta(B)} \text{ for } B \in E(u, v). \tag{32}$$

With one state node per hyperedge layer that contains the node, the multilayer representation requires the most nodes and links to describe the walk. Variations include walks that are biased to similar hyperedges [26].

These network representations and random-walk models have different advantages for modeling multi-body interactions [25, 26]. The bipartite network representation enables the most efficient community detection in terms of computation time because it uses the fewest links and often results in shallower solutions. Multilayer network representations reinforce flows within similar layers, giving the deepest modular structures with the most overlap. However, this increase in detected modules comes at a high computational cost, as combining fully connected layers with other layers requires more nodes and links than the bipartite network representation. If the research question does not call for hyperedge assignments or overlapping modules, the unipartite network representation offers a tradeoff with intermediate compactness, speed, and the ability to reveal modular regularities. Among random-walk models, lazy walks typically give more modules in deeper nested structures, while non-lazy walks result in higher modular overlap.

6 Networks with Node Attributes

Nodes in real-world networks are often labeled with additional information, such as a person's age, an animal's species, an object's shape, or, more generally, as belonging to a certain category. We can characterize the dynamic process that operates on such an annotated network better by incorporating node-type information in the flow modeling or flow mapping step.

6.1 Networks with Metadata

To combine additional metadata with the network structure, we can modify the flow mapping step by extending the map equation with an additional metadata codebook [23] or the flow modeling step by choosing which random-walker steps to encode depending on the metadata [4]. Alternatively, we can use metadata as a part of an empirical prior for a Bayesian estimate of the random walker's transition rates [79] (see Section 7).

For simplicity, we assume unipartite networks and categorical metadata labels and denote node u's label by ℓ_u. Furthermore, let $\mathcal{L}$ be the set of possible metadata labels.

The content map equation. In addition to communicating the random walker's current node, the content map equation requires encoding the current node's metadata label [23]. Given a partition M, the metadata label ℓ appears with frequency

$$r_{\mathsf{m},\ell} = \sum_{u \in \mathsf{m}, \ell_u = \ell} p_u \tag{33}$$

within module $\mathsf{m} \in \mathsf{M}$. The total metadata weight of module m is

$$r^{\circlearrowleft}_{\mathsf{m}} = \sum_{\ell \in \mathcal{L}} r_{\mathsf{m},\ell} = \sum_{u \in \mathsf{m}} p_u. \tag{34}$$

The entropy for encoding the metadata labels for the nodes along the random walker's trajectory in module m is

$$H(R_{\mathsf{m}}) = -\sum_{\ell \in \mathcal{L}} \frac{r_{\mathsf{m},\ell}}{r^{\circlearrowleft}_{\mathsf{m}}} \log_2 \frac{r_{\mathsf{m},\ell}}{r^{\circlearrowleft}_{\mathsf{m}}}, \tag{35}$$

where R_{m} is the set of metadata frequencies in module m. Combining Equation (35) with Equation (11) gives the so-called content map equation,

$$L(\mathsf{M}) = q_{\curvearrowleft} H(Q) + \sum_{\mathsf{m} \in \mathsf{M}} p^{\circlearrowleft}_{\mathsf{m}} H(P_{\mathsf{m}}) + \eta \sum_{\mathsf{m} \in \mathsf{M}} r^{\circlearrowleft}_{\mathsf{m}} H(R_{\mathsf{m}}), \tag{36}$$

where η is a parameter to control the metadata entropy's weight.

The content map equation produces modules with low metadata entropy. It favors splitting structural modules into sub-modules based on their metadata labels and preserving structural boundaries: nodes with the same metadata label remain separated if they belong to different structural modules.

Metadata-dependent encoding of random walks. To map nonlocal relationships between metadata and network structure, we use a metadata-dependent encoding of random walks [4]. For a random walker starting at node u and stepping to node v, we encode the step with probability $\epsilon(\ell_u, \ell_v)$, which depends on the metadata labels at the start node u and the currently visited node v, or the random walker continues without encoding to another node, say v'. We encode the step to v' with probability $\epsilon(\ell_u, \ell_{v'})$, or the random walker continues in the same fashion until we eventually encode the step from the start node u to the currently visited node. After encoding, the random walker restarts at a random node u' chosen proportionally to the node's visit rate.

Varying the coding this way, we derive a metadata-dependent encoding graph from a given network: The nodes are the same as in the original network, and the links derive from the coding behavior, which can connect disconnected nodes in the original network. The link

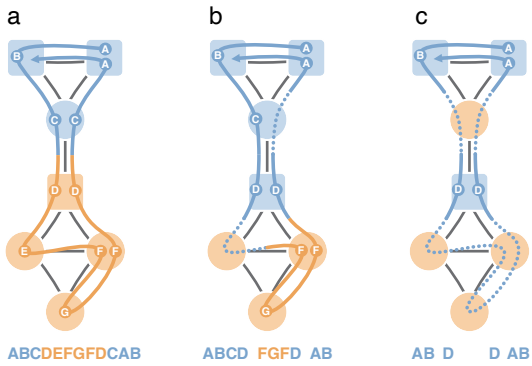

Fig. 12. **Random walks with metadata-dependent encoding probabilities.** In this schematic example with a single, long random walk, we encode the random walker's next step if the target node's metadata are the same as for the previously encoded node. When the metadata differ, encoding with a probability between $\frac{1}{3}$ and 1 gives the solution in (a), between 0.1 and $\frac{1}{3}$ the solution in (b), and less than 0.1 the solution in (c). Node shapes represent metadata, and node colors optimal partitions. The random walks are colored by the currently encoded module and labeled with circles when they encode a transition and dotted lines when they skip nodes. The alphabetic codes represent the walks with this metadata-dependent encoding scheme. Figure adapted from Ref. [4].

weights correspond to the fractions of time that we encode each respective link, such that the weight of link (u, v) is the number of times that we encode a step to v after restarting at u, divided by the total number of encoded steps. Effectively, this yields a metadata-dependent transition matrix and metadata-dependent visit rates, which we use to detect modules with Infomap. Unlike the content map equation, modeling metadata-dependent flow enables grouping nodes that belong to different structural modules in the original network.

The encoding probability function determines what specific patterns the metadata-dependent encoding graph captures (Figure 12). For example, $\epsilon(\ell_u, \ell_v) = 1$ forces the walker to encode every step regardless of metadata labels, equivalent to the standard map equation. Setting $\epsilon(\ell_u, \ell_v) = \delta_{\ell_u,\ell_v}$, where δ is the Kronecker delta, encodes only on nodes with the same metadata labels, splitting the network into modules purely based on metadata while ignoring link patterns. For binary metadata labels, a versatile choice is

$$\epsilon(\ell_u, \ell_v) = p\delta_{\ell_u,\ell_v} + \frac{p}{c}\left(1 - \delta_{\ell_u,\ell_v}\right), \tag{37}$$

where $p \in [0, 1]$ and $c \in [p, \infty]$. Setting $c > 1$ gives assortative, and setting $p < c < 1$ gives disassortative modules. For $c = 1$, the expression simplifies to $\epsilon(\ell_u, \ell_v) = p$ without metadata dependence. The metadata-dependent encoding probability function can also take real-valued or vectorial metadata labels [4].

While generating the metadata-dependent random walk transition graph requires additional computation, the method enables capturing nonlocal interactions, including different metadata types, and modeling different metadata roles.

6.2 Bipartite Networks

The standard map equation can identify modules in bipartite networks, ignoring their two distinct node types. But by treating these node types, left nodes V_L and right nodes V_R, as categorical attributes and making the map equation aware of them, we can exploit the bipartite network's constrained structure to compress random walks more efficiently. A bipartite coding scheme captures the alternating pattern of visits between left and right nodes, and the bipartite map equation [7] improves compression by using separate codebooks for left-to-right and right-to-left transitions,

$$L_B(\mathsf{M}) = q^R_{\curvearrowright} H\left(Q^R\right) + \sum_{\mathsf{m}\in\mathsf{M}} p^{R\circlearrowleft}_{\mathsf{m}} H\left(P^R_{\mathsf{m}}\right) + q^L_{\curvearrowright} H\left(Q^L\right) + \sum_{\mathsf{m}\in\mathsf{M}} p^{L\circlearrowleft}_{\mathsf{m}} H\left(P^L_{\mathsf{m}}\right). \tag{38}$$

Here, Q^R and Q^L are the left-to-right and right-to-left module entry rates, respectively; P^R_{m} and P^L_{m} are the sets of right and left node visit rates in module m, including the left-to-right and right-to-left module exit rates, respectively; and $q^R_{\curvearrowright}$ and $q^L_{\curvearrowright}$ are the rates at which the left-to-right and right-to-left index level codebooks are used, respectively.

To control at which rate the bipartite node types are used, we introduce a node-type forgetting parameter α. Assuming that we misremember node types with probability α, the available amount of information about node types is $I = 1 - H(\alpha, 1-\alpha)$ bits. Because of this uncertainty about node types, we interpret node visit rates as pairs of left and right flow, leading to mixed visit rates $p_{\alpha,u} = ((1-\alpha)p_u, \alpha p_u)$ for left nodes $u \in V_L$, and $p_{\alpha,v} = (\alpha p_v, (1-\alpha)p_v)$ for right nodes $v \in V_R$. The bipartite map equation with varying node-type memory for two-level partitions, M, is

$$L_\alpha(\mathsf{M}) = q_{\alpha,\curvearrowright} H(Q_\alpha) + \sum_{\mathsf{m}\in\mathsf{M}} p^{\circlearrowleft}_{\alpha,\mathsf{m}} H\left(P_{\alpha,\mathsf{m}}\right), \tag{39}$$

where $q_{\alpha,\curvearrowright}$ is the mixed rate at which the index codebook is used, Q_α is the set of mixed module entry rates, and $p^{\circlearrowleft}_{\alpha,\mathsf{m}}$ is the mixed rate at which module m's codebook is used [7]. As before, we can generalize Equation (39) to multilevel partitions through recursion. We recover the standard map

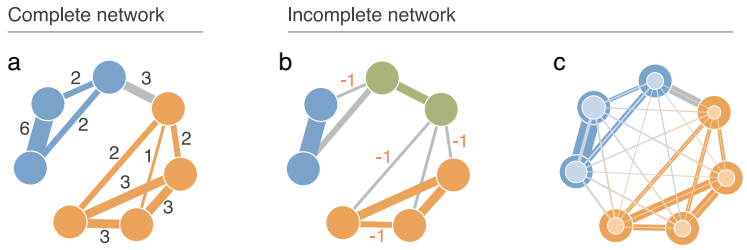

Fig. 13. **Community detection in an undirected weighted network with complete and missing link observations.** (a) The map equation identifies two modules in a complete network with accurate network flows. (b) Missing link observations introduce inaccuracies leading to overfitting. (c) The map equation with regularized network flows recovers the complete network's community structure. Colors represent community assignments. The width of the lighter lines represents prior link weights, and the size of the lighter node centers indicates prior transition probability. Figure adapted from Ref. [79].

equation (Equation (11)) for $\alpha = \frac{1}{2}$, and the bipartite map equation (Equation (38)) for $\alpha \in \{0, 1\}$, where setting $\alpha = 1$ flips the node types. By using node-type information at rates between $I = 0$ bits and $I = 1$ bit, we change the map equation's resolution and detect communities at different scales.

By varying the amount of available node-type information, I, the bipartite map equation reveals the organizational structure of bipartite networks at different resolutions [7].

7 Networks with Incomplete Data

In the previous sections, we discussed how to take advantage of the available data and identify flow models that best describe modular regularities. Once we have chosen a flow model, the standard map equation estimates the stationary flow distribution based on what it sees in the data. However, data can be noisy with incomplete link measurements, leading to inaccurate estimates of the actual stationary flow distribution in the system. Consequently, when missing observations distort the balance between module and index-level codebooks, the map equation can overfit to the noise and highlight spurious communities [33, 78, 79].

To address this challenge, we can extend the map equation to account for incomplete data by applying a Bayesian method to regularize the random walker's transition rates (Figure 13) [79]. This method allows us to account for potentially missing connections in the observed network. It combines the observed connections with prior expectations, effectively reducing the effects of sampling noise.

7.1 Bayesian Estimate of Transition Rates

We assume that a random walker steps from node u to nodes v proportional to $\tilde{p}_u = (\tilde{p}_{uv_1}, \ldots, \tilde{p}_{uv_N})$. The link weights $(w_{uv_1}, \ldots, w_{uv_N})$ represent a sample of the hidden distribution $\tilde{p}_u$. We assume non-negative integer weights for simplicity, but the method also works for non-negative real weights. If we interpret the transition probability p_{uv} given by Equation (1) as a maximum likelihood estimate of $\tilde{p}_{uv}$, p_{uv} can deviate significantly from $\tilde{p}_{uv}$ in the case of noisy data. To reduce the effects of statistical fluctuations, we estimate the average posterior transition rates

$$\hat{p}_{uv}(w) = \int \tilde{p}_{uv} P(\tilde{p}_u | w_{uv_1}, \ldots, w_{uv_N}) d\tilde{p}_u, \tag{40}$$

where $P(\tilde{p}_u | w_{uv_1}, \ldots, w_{uv_N})$ is a posterior over the unknown distribution $\tilde{p}_u$ given by Bayes' rule. As prior distribution $P(\tilde{p}_u)$, we choose the Dirichlet distribution, which is the conjugate prior of

the multinomial distribution and enables analytical calculations:

$$P(\tilde{p}_u|\gamma_u) = \frac{\Gamma(\gamma_{uv_1} + \cdots + \gamma_{uv_N})}{\Gamma(\gamma_{uv_1}) \ldots \Gamma(\gamma_{uv_N})} \prod_{v \in V} \tilde{p}_{uv}^{\gamma_{uv}-1}. \tag{41}$$

$\Gamma(x)$ is the gamma function, and $\gamma_{uv_1}, \ldots, \gamma_{uv_N}$ are parameters of the distribution that reflect our prior assumption about links before we observe the network data. After integrating Equation (40), we obtain

$$\hat{p}_{uv} = (1 - \alpha_u)\frac{w_{uv}}{\sum_v w_{uv}} + \alpha_u \frac{\gamma_{uv}}{\sum_v \gamma_{uv}}, \tag{42}$$

where

$$\alpha_u = \frac{\sum_{v \in V} \gamma_{uv}}{\sum_{v \in V} w_{uv} + \gamma_{uv}}. \tag{43}$$

The Bayes estimate of the transition rates in Equation (42) represents a weighted average of the empirical transition probabilities and the prior expectations. As the number of observed out-links of node u increases, the estimate approaches the empirical probabilities, whereas sparse observations cause the prior parameters to dominate the estimate. Thus, the quality of this Bayesian estimate depends both on the number of link observations and on the choice of the prior parameters γ_{uv}.

7.2 Prior Parameter

When selecting prior parameters γ, we aim to ensure that the map equation identifies meaningful communities when enough data are provided, while avoiding overfitting in undersampled networks. In the limit of very sparse data, a desirable outcome is a single community encompassing all nodes, reflecting that the available observations do not support finer partitions.

We treat link probabilities and link weights as decoupled and write

$$\gamma_{uv} = \lambda_{uv} c_{uv}, \tag{44}$$

where λ_{uv} is a connectivity parameter that denotes the prior probability that nodes u and v are connected, and the weight parameter c_{uv} determines prior link weights.

Prior link weights. We use the so-called continuous configuration model [60] to obtain an empirical estimate for the prior link weights,

$$c_{uv} = \frac{\sum_{n \in V} k_n^{\text{in}} + k_n^{\text{out}}}{\sum_{n \in V} s_n^{\text{in}} + s_n^{\text{out}}} \frac{s_u^{\text{out}} s_v^{\text{in}}}{k_u^{\text{out}} k_v^{\text{in}}}, \tag{45}$$

where k_u^{in} and k_u^{out} denote observed in- and out-degrees, and $s_u^{\text{in}} = \sum_v w_{vu}$ and $s_u^{\text{out}} = \sum_v w_{uv}$ denote in- and out-strengths for node u. The continuous configuration model preserves the expected weights of in- and out-links. For unweighted networks, where $s_u = k_u$, it assigns weights $c_{uv} = 1$ to all links.

General connectivity parameter. To reduce the bias induced by incomplete observations, we assume an uninformative connectivity corresponding to a network without modular structure, where nodes u and v are connected with uniform probability. If no information about node attributes is provided, we use

$$\lambda_{uv} = \lambda = \frac{\ln N}{N}. \tag{46}$$

This value for λ is the theoretical threshold at which Erdős–Rényi random networks become almost surely connected. When $\lambda < \frac{\ln N}{N}$, the prior network has disconnected components and can fail to reduce overfitting, while $\lambda > \frac{\ln N}{N}$ can be too strong and wash out modular regularities [78, 79].

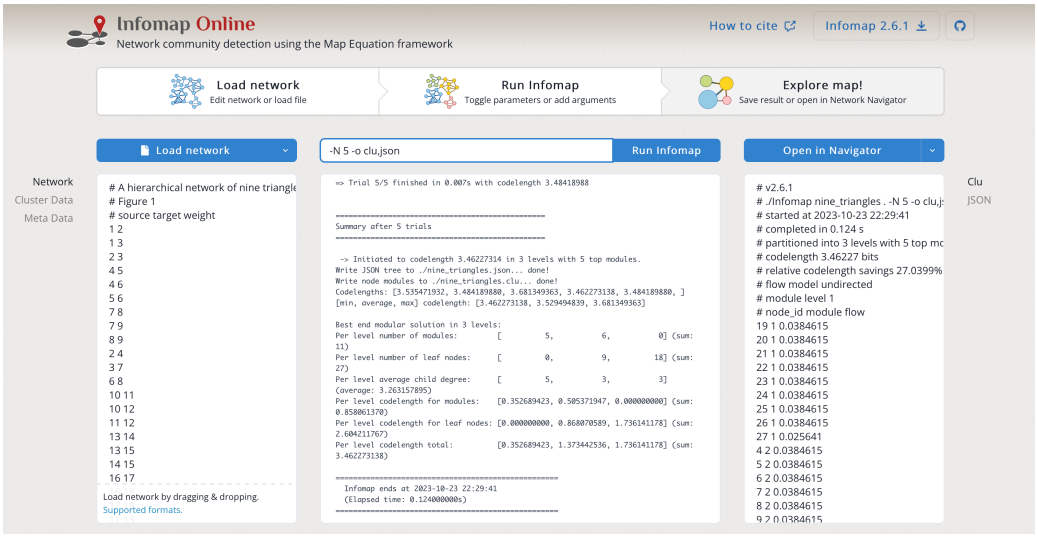

Fig. 14. **Screenshot of Infomap Online**. Infomap Online is a client-side web application that enables users to run Infomap in the web browser. We achieve this by compiling Infomap from C++ to JavaScript and providing a graphical user interface for all input and output parameters. Infomap Online is available at https://www.mapequation.org/infomap/.

Bipartite connectivity parameter. If the given network has a bipartite structure with N_L left nodes and N_R right nodes, we adjust the connectivity parameter accordingly: we use the smallest λ^{bi} at which a random bipartite network is almost surely connected [73]

$$\lambda^{\text{bi}} = \frac{\ln(N_L + N_R)}{\min(N_L, N_R)}. \tag{47}$$

We also adapt the definition of prior link weights to reflect that links in bipartite networks must connect different-type nodes,

$$\gamma_{uv}^{\text{bi}} = \left(1 - \delta_{\tau_u \tau_v}\right) \lambda^{\text{bi}} c_{uv}, \tag{48}$$

where τ_u and τ_v are the types of nodes u and v, that is, left or right.

Metadata connectivity parameter. Nodes annotated with metadata allow further fine-tuning the connectivity parameter. We expect the N_ℓ nodes with the same metadata label ℓ to be connected with higher probability. With metadata labels ℓ_u and ℓ_v for nodes u and v, we adjust the prior connectivity to

$$\lambda_{uv}^{\text{meta}} = \lambda + \delta_{\ell_u \ell_v} \lambda_{\ell_u}, \tag{49}$$

where δ is the Kronecker delta, and $\lambda_{\ell_u} = \frac{\ln N_{\ell_u}}{N_{\ell_u}}$.

8 Software and Visualization

Infomap is the search algorithm and a key component of the map equation framework. But to understand the organization of large complex networks, we require both efficient algorithms and powerful visualizations. We have developed several visualization tools with different purposes and made them available as web applications for anyone to use. The first step of community detection with the map equation framework is to install or get access to Infomap.

8.1 Infomap

Infomap is the name of the search algorithm that optimizes the map equation. Its core is written in C++ and requires a compiler with C++-14 support.

Python package. To support the many researchers that use Python for their community detection pipelines, we made Infomap available as a Python package, also called `infomap` [20]. This package is a small wrapper around the core C++ version of Infomap with additional helper functions and integrates with other popular packages, such as `networkx` and `pandas`. A basic version of the package is also included in the popular community detection library CDLib [68].

Infomap Online. While the most computationally efficient community detection pipeline is to run the stand-alone C++ or Python versions of Infomap, we have embedded a JavaScript version of Infomap into a web application we call Infomap Online (Figure 14) [41]. This embedded Infomap version supports the same network inputs as C++ Infomap. Infomap Online also serves as interactive documentation for Infomap with examples and simple network visualizations.

8.2 Map Equation Demo

While the map equation does not depend on explicitly deriving codewords or simulating random walks, visualizing the modular description of a random walk can highlight the mechanics of the map equation. To do this, we have developed a web-based demo application [42]. In this application, we show how modular Huffman codes [45] enable re-using codewords in different modules by encoding module transitions with exit and enter codewords. This enables a shorter expected codelength for partitions where the random walker spends a relatively long time inside modules compared to exiting and entering other modules. In Figure 15, we show a realization of a random walk starting in the green—top right—module and transitioning to the orange—bottom left—module. Every module transition is encoded using a module-specific exit codeword followed by an enter codeword from the index codebook.

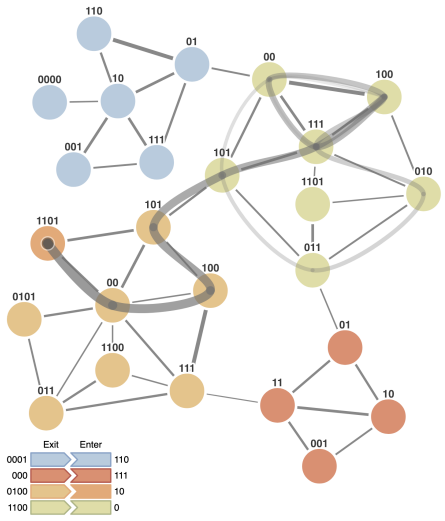

Fig. 15. **Modular description of a random walk on a network.** Each module has its own codebook comprised of node and exit codewords. A global index codebook is comprised of enter codewords for all modules. Each codeword is selected using the Huffman code algorithm on the possible transitions in each codebook.

8.3 Infomap Network Navigator

Understanding real-world systems relies on effectively mapping complex networks. To comprehend and navigate the intricate multilevel communities within these networks, it is crucial to employ powerful visualizations that can transform textual representations into visually intuitive maps. To address this need, we have developed an interactive web application called Infomap Network Navigator [24, 40]. This tool allows users to visualize and explore multilevel maps of both conventional and higher-order networks in a manner similar to using mapping software such as Google Maps.

We visualize multilevel communities using top-level modules represented by circles. The circle areas indicate the contained flow volume, while the border thickness represents the exiting flow volume (inset in Figure 16). Modules are connected by aggregated links, with their thickness and

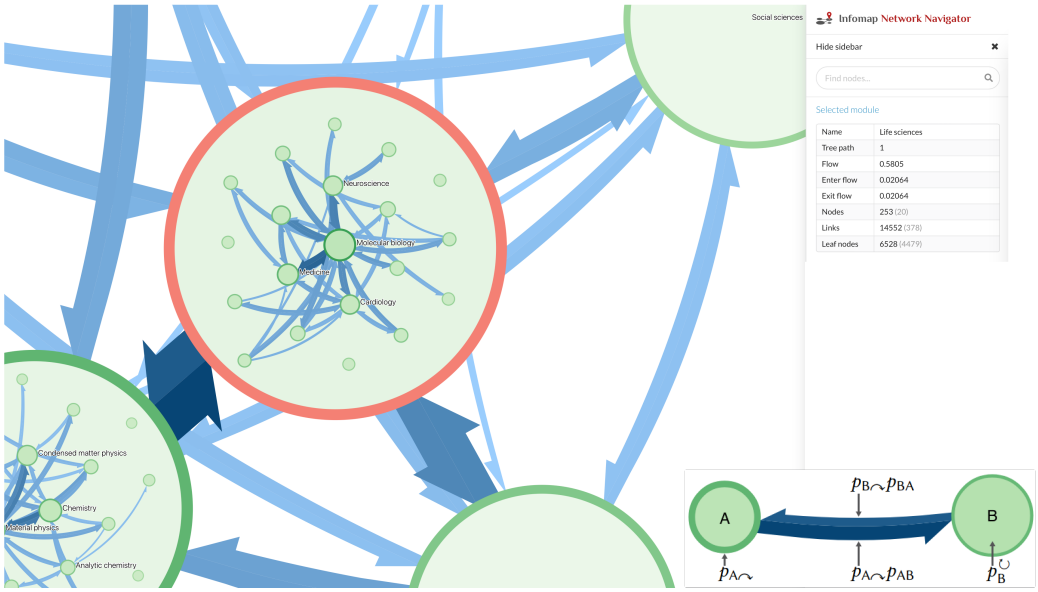

Fig. 16. **Infomap Network Navigator.** The Network Navigator is a web application that generates an interactive zoomable map for networks clustered with Infomap. Main panel: The citation flow patterns in science are organized in a multilevel partition. Zooming in reveals sub-modules within the life sciences, such as molecular biology and medicine. Inset: Schematic representation of two connected modules. Modules are represented by circles with areas proportional to the contained flow $p_{\mathrm{m}\circlearrowleft}$ and border thickness proportional to the exiting flow $p_{\mathrm{m}\curvearrowright}$. Links with thickness and lightness proportional to—and length inversely proportional to—the aggregated flow volume between modules $p_{m\curvearrowright}p_{mn}$. The Infomap Network Navigator is available at https://www.mapequation.org/navigator/.

color lightness reflecting the flow between them. Additionally, we adjust the link lengths inversely proportional to the inter-module flow, emphasizing varying connection strengths. Figure 16 illustrates using the Infomap Network Navigator to highlight multilevel citation flow patterns in science. At the top level, the application visualizes research fields like the life sciences, physical sciences, and social sciences. By zooming in using the mouse wheel or trackpad, more detailed information becomes visible. For example, within the life sciences, the application reveals further divisions into research areas such as molecular biology and medicine.

8.4 Alluvial Diagram Generator

Complex systems, such as social or biological systems, are dynamic. This results in changing interaction patterns over time. Understanding this change is essential to gain insights into the organization and evolution of complex systems.

Various summary statistics can quantify these structural changes, such as variation of information or normalized mutual information, which compute a distance or similarity between two partitions [27, 37, 52, 59]. While these metrics have their use cases, they do not capture essential information about how the networks change.

Powerful visualization tools are necessary to effectively visualize and understand how and where these changes occur. Alluvial diagrams visually represent the changing organization of networks by embedding each network's modules as vertically stacked blocks (Figure 17) [44, 70]. The height of these blocks is proportional to the flow volume of the nodes they contain. By positioning different

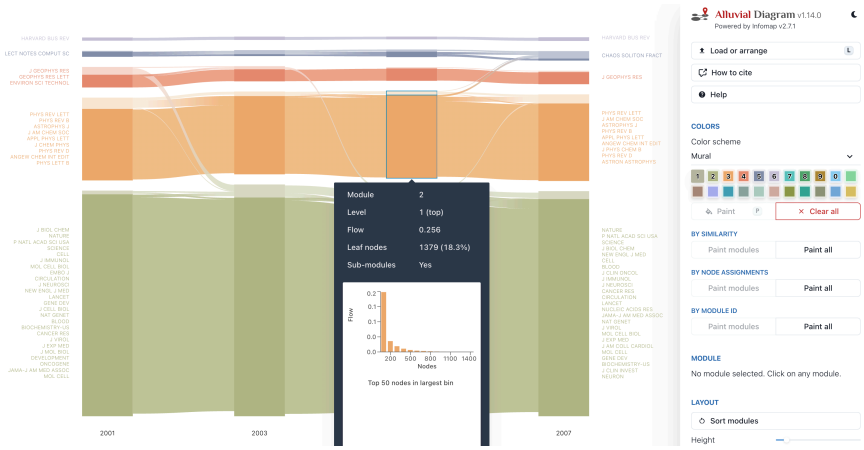

Fig. 17. **Screenshot of the Alluvial Diagram Generator**. Alluvial diagrams visualize how the modular organization of networks changes. Each vertical stack represents the network's organization, here for journal citation data from different time points. The Alluvial Diagram Generator is available at https://www.mapequation.org/alluvial/.

networks adjacent to one another and connecting modules with shared nodes using streamfields—the middle part of Figure 18(b)—the diagrams highlight the changing organization. To incorporate multilevel solutions in alluvial diagrams, we display hierarchically nested sub-modules above their corresponding super-modules. We illustrate this in the right stack in Figure 18(b).

We developed an alluvial diagram generator as a web application [43] that operates within the user's web browser, ensuring that data never leave the local machine. This client-side architecture is crucial for researchers working with sensitive data.

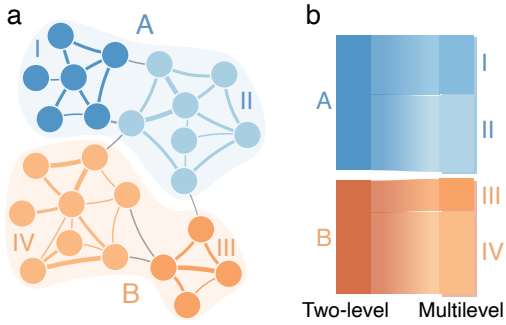

Fig. 18. **Schematic alluvial diagram of multilevel network structure.** (a) A weighted network with a multilevel modular structure. The two blue and orange top-level modules (A–B) are further divided into two sub-modules each (I–IV), indicated by color lightness. (b) An alluvial diagram representation of the top-level and multilevel modules in (a) using the same colors. Columns of blocks represent modules with heights proportional to the contained flow volume. The leftmost column is an ordinary two-level alluvial diagram representation. The multilevel representation to the right shows multiple levels, with the background showing the top-level organization. Streamfields connect modules in the left and right columns that share nodes.

9 Applications

Applications in network science range from fundamental tasks such as assessing node centrality and similarity to more advanced approaches, including bioregionalization and model selection for correlational data. These problems are often tackled with heuristic methods that provide practical but ad hoc solutions. By capitalizing on the map equation's coding principles, we can instead ground them in a principled, information-theoretic framework. With Infomap, this foundation translates into efficient and robust methods that support reliable applications across fields from biodiversity research to systems biology.

9.1 Map Equation Centrality

Node centrality measures characterize how important or influential each node in a network is. Traditional measures take either microscopic or macroscopic perspectives: node degree centrality counts local connections, while PageRank tracks global flows, for example. Yet these measures overlook the mesoscopic community structure. Degree centrality cannot distinguish between a node embedded deep within a community and a bridge node if they have the same number of connections, just as PageRank cannot, if they have identical visit rates.

Drawing on the map equation's coding principles, map equation centrality offers an information-theoretic approach to quantifying community-aware node importance. Similar to how certain auctions calculate the winner's price as the total harm it causes other bidders [53, 83], map equation centrality quantifies a node's importance as the collective benefit to other nodes if that node were removed from the coding scheme. Specifically, how many bits shorter would the codewords for remaining nodes become without having to encode visits to the target node [6].

Given a partition M, map equation centrality for node u in module m_u can be expressed as

$$\lambda\left(\mathsf{M}, u\right) = -\left(p^{\circlearrowleft}_{\mathsf{m}_u} - p_u\right) \log_2 \frac{p^{\circlearrowleft}_{\mathsf{m}_u} - p_u}{p^{\circlearrowleft}_{\mathsf{m}_u}}. \tag{50}$$

Because the map equation uses modular codebooks, removing a node only affects codewords within its own module, making the centrality calculation locally efficient.

In networks with clear community structure, map equation centrality reveals patterns invisible to traditional measures. For example, bridge nodes connecting communities may have modest degree or PageRank, yet their removal can drastically decrease coding costs for remaining nodes in the module, revealing their centrality from a community-aware perspective.

9.2 Map Equation Similarity

Node similarity measures quantify how similar two nodes are, enabling representation learning and link prediction in networks. Recent approaches learn features from local subgraphs, overlapping neighborhoods, or shortest paths to embed nodes in low-dimensional Euclidean spaces [38]. But symmetric distance measures cannot distinguish directed relationships where similarity from u to v differs from v to u, and low-dimensional representations struggle to capture complex, non-metric relationships in networks.

Leveraging the map equation's coding principles, map equation similarity—*mapsim* for short—sidesteps these geometric constraints [8]. Instead of embedding nodes in metric space, mapsim grounds similarities in compression efficiency within the network's modular structure. While networks constrain random walks to existing links, the map equation's coding scheme can describe transitions between any node pair. Once we know a network's community structure and its corresponding codewords, we can calculate the description length for any potential transition, whether the link exists or not.

To calculate how similar node u is to node v given partition M, mapsim (u, v, M), we find the smallest module $\mathsf{m} \in \mathsf{M}$ that contains both u and v. The similarity equals the product of the rate $r_{u,\mathsf{m}}$ at which a random walker at u transitions up to the root level of m and the rate $r_{\mathsf{m},v}$ at which a random walker at the index level of m transitions to and visits node v, mapsim $(u, v, \mathsf{M}) = r_{u,\mathsf{m}} r_{\mathsf{m},v}$. We interpret mapsim (u, v, M) as the distance from u to v, in bits, by setting $d_{uv} = \log_2$ mapsim (u, v, M).

This information-theoretic, community-based approach proves effective: across 47 networks, mapsim achieved average performance more than 7 percent higher than its closest competitor [8].

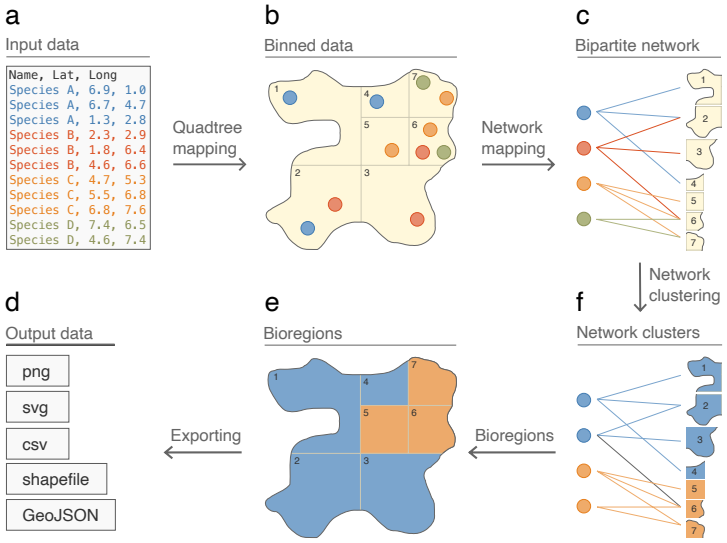

Fig. 19. **Infomap Bioregions**. Schematic illustration of bioregional mapping from species distribution data to bioregions. Network-based methods identify bioregions from species distribution data (a) by binning species observations into spatial grid cells (b). Grid cells and species form nodes in a bipartite network where a links indicate species occurrence in grid cells (c). Infomap detects communities (d). In bipartite networks, communities contain grid cells and species; the grid cells assigned to the same community form a bioregion (e). In the interactive Infomap Bioregions tool, available at https://www.mapequation.org/bioregions/, bioregional maps can be exported to various formats (f).

9.3 Infomap Bioregions

In biodiversity research, data-derived biogeographical regions, or simply bioregions, reveal how species are grouped at large spatial scales. They serve as essential units for understanding historical biogeography, ecology, and evolution, and for identifying the areas most critical for conservation. Traditional approaches rely on subjective clustering of similarity measures, often missing the complex co-occurrence patterns that define ecological communities.

Infomap Bioregions grounds bioregional delimitation in modular compression of network flows [19]. The method first bins all species into spatial grid cells. These species and grid cells form a bipartite network, where links connect species to grid cells where they occur. Infomap then identifies bioregions by detecting network modules where species assemblages remain distinct, revealing natural boundaries in biological diversity (Figure 19). The interactive web application allows researchers to upload occurrence data, tune parameters, and export results for further analysis (Figure 20)

This principled approach has proven effective for conservation biologists. They have used Infomap Bioregions to delineate bioregions in one of our planet's most biodiverse hotspots, the tropical Andes, to understand the spatial and temporal evolution of the biota [39]. In another study, researchers used this approach to analyze about 25,000 vascular plant species in tropical Africa, determining whether different forms of plant growth display similar diversity patterns [17].

9.4 Model Selection with Correlational Data

Correlational data represented as networks underpin analyses across the sciences. Identifying modules in these networks can reveal genes with shared functions or species that prefer similar

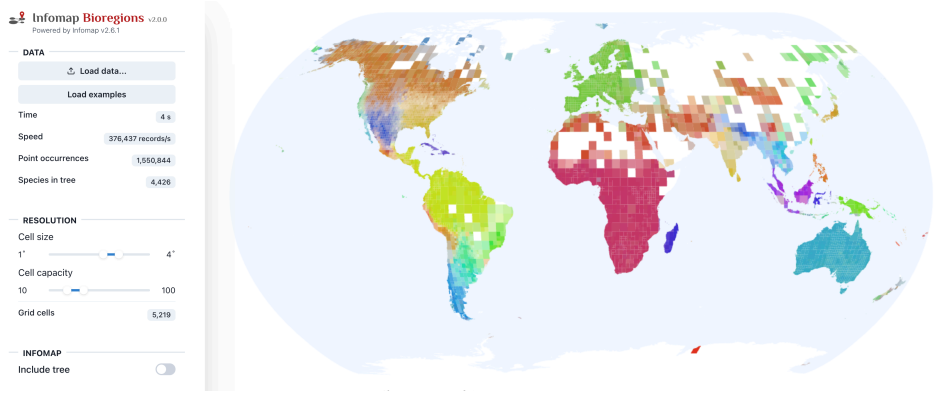

Fig. 20. **Screenshot of Infomap Bioregions**. The web application enables interactive exploration of species distribution data and bioregional patterns. The map shows mammal occurrence distributions where white areas lack sufficient data for statistics. Infomap Bioregions is available at https://www.mapequation.org/bioregions.

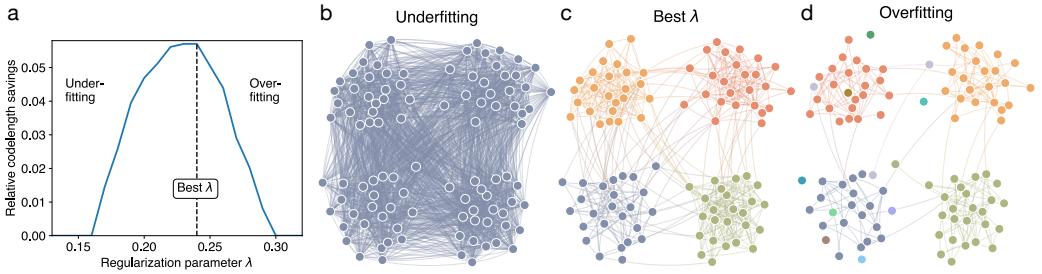

Fig. 21. **Model selection with the map equation.** We can use the map equation to find the best network model of correlational data or other similarity data. The relative codelength savings show how much the model's description length is compressed as we vary regularization strength, comparing the best identified partition to a one-module solution. The codelength savings peak at the optimal model (a), which best balances under- and overfitting modular structure to the data, as illustrated by weak (b), optimal (c), and strong (d) regularization.

conditions. But they are typically dense and noisy, making it difficult to distinguish meaningful structure from statistical artifacts. To extract useful information from the correlation networks, researchers must regularize them, but common approaches based on heuristics or likelihood maximization risk either erasing structure by underfitting or generating spurious patterns by overfitting.

The map equation offers a more principled approach (Figure 21). By aligning model selection directly with the goal of identifying modular structure, it replaces arbitrary thresholding with a direct measure of clustering performance. Specifically, the map equation's maximum codelength savings determine the regularization strength. This balance between complexity and fit helps identify only those modules that are robust to noise and supported by data [56–58]. This principle extends to Gaussian graphical models. A module-based graphical lasso infers sparse precision matrices from correlational data with improved performance under noise [56].

Applied to gene co-expression data, the method uncovers modular structure that remain hidden with standard techniques such as WGCNA or the graphical lasso [56–58].

10 Conclusion

Simplifying the dynamic processes on networks to uncover how complex systems work remains a challenging problem with many applications. To help researchers take full advantage of the map equation framework and its community-detection algorithm Infomap, we explain the information-theoretical principles of the map equation and summarize its many generalizations to various network representations, flow models, and modular descriptions. As network science advances, flow-based community detection methods will remain essential for unraveling the complexities of networked systems and their inner workings. We hope our review can inspire further generalizations of the map equation to simplify and highlight important structures in innovative network models.

Data and Code Availability

Infomap is available at https://mapequation.org/infomap and its source code at https://github.com/mapequation/infomap. Data and notebooks are available at https://github.com/mapequation/infomap-tutorial-notebooks.

Author Contributions

All authors wrote and edited the manuscript.

Competing Interests

The authors declare that they have no competing interests.

References

[1] Rodrigo Aldecoa and Ignacio Marín. 2013. Exploring the limits of community detection strategies in complex networks. *Scientific Reports* 3, 1 (2013), 2216.
[2] Ulf Aslak, Martin Rosvall, and Sune Lehmann. 2018. Constrained information flows in temporal networks reveal intermittent communities. *Physical Review E* 97, 6 (2018), 062312.
[3] Albert-László Barabási and Márton Pósfai. 2016. *Network Science*. Cambridge University Press, Cambridge. Retrieved from http://barabasi.com/networksciencebook/
[4] Aleix Bassolas, Anton Holmgren, Antoine Marot, Martin Rosvall, and Vincenzo Nicosia. 2022. Mapping nonlocal relationships between metadata and network structure with metadata-dependent encoding of random walks. *Science Advances* 8, 43 (2022), eabn7558.
[5] Federico Battiston, Giulia Cencetti, Iacopo Iacopini, Vito Latora, Maxime Lucas, Alice Patania, Jean-Gabriel Young, and Giovanni Petri. 2020. Networks beyond pairwise interactions: Structure and dynamics. *Physics Reports* 874 (2020), 1–92.
[6] Christopher Blöcker, Juan Carlos Nieves, and Martin Rosvall. 2022. Map equation centrality: Community-aware centrality based on the map equation. *Applied Network Science* 7, 1 (2022), 56.
[7] Christopher Blöcker and Martin Rosvall. 2020. Mapping flows on bipartite networks. *Physical Review E* 102, 5 (2020), 052305.
[8] Christopher Blöcker, Jelena Smiljanić, Ingo Scholtes, and Martin Rosvall. 2022. Similarity-based link prediction from modular compression of network flows. In *Proceedings of the 1st Learning on Graphs Conference*.
[9] Christopher Blöcker, Chester Tan, and Ingo Scholtes. 2024. The map equation goes neural: Mapping network flows with graph neural networks. In *Proceedings of the Advances in Neural Information Processing Systems*.
[10] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (2008), P10008.
[11] Ludvig Bohlin, Daniel Edler, Andrea Lancichinetti, and Martin Rosvall. 2014. Community detection and visualization of networks with the map equation framework. In *Proceedings of the Measuring Scholarly Impact*. Springer, 3–34.
[12] Joaquín Calatayud, Rubén Bernardo-Madrid, Magnus Neuman, Alexis Rojas, and Martin Rosvall. 2019. Exploring the solution landscape enables more reliable network community detection. *Physical Review E* 100, 5 (2019), 052308.
[13] Timoteo Carletti, Federico Battiston, Giulia Cencetti, and Duccio Fanelli. 2020. Random walks on hypergraphs. *Physical Review E* 101, 2 (2020), 022308.
[14] Uthsav Chitra and Benjamin Raphael. 2019. Random walks on hypergraphs with edge-dependent vertex weights. In *Proceedings of the 36th International Conference on Machine Learning*. PMLR, 1172–1181.

[15] Manlio De Domenico, Andrea Lancichinetti, Alex Arenas, and Martin Rosvall. 2015. Identifying modular flows on multilayer networks reveals highly overlapping organization in interconnected systems. *Physical Review X* 5, 1 (2015), 011027.
[16] Jean-Charles Delvenne, Sophia N. Yaliraki, and Mauricio Barahona. 2010. Stability of graph communities across time scales. *Proceedings of the National Academy of Sciences* 107, 29 (2010), 12755–12760.
[17] Vincent Droissart, Gilles Dauby, Olivier J. Hardy, Vincent Deblauwe, David J. Harris, Steven Janssens, Barbara A. Mackinder, Anne Blach-Overgaard, Bonaventure Sonké, Marc SM Sosef, et al. 2018. Beyond trees: Biogeographical regionalization of tropical Africa. *Journal of Biogeography* 45, 5 (2018), 1153–1167.
[18] Daniel Edler, Ludvig Bohlin, and Martin Rosvall. 2017. Mapping higher-order network flows in memory and multilayer networks with infomap. *Algorithms* 10, 4 (2017), 112.
[19] Daniel Edler, Thaís Guedes, Alexander Zizka, Martin Rosvall, and Alexandre Antonelli. 2017. Infomap bioregions: Interactive mapping of biogeographical regions from species distributions. *Systematic Biology* 66, 2 (2017), 197–204.
[20] Daniel Edler, Anton Holmgren, and Martin Rosvall. 2020. The Infomap Python package. Retrieved from https://mapequation.github.io/infomap/python/
[21] Daniel Edler, Anton Holmgren, and Martin Rosvall. 2022. The MapEquation software package. Retrieved from https://mapequation.org
[22] Daniel Edler, Jelena Smiljanić, Anton Holmgren, Alexandre Antonelli, and Martin Rosvall. 2022. Variable Markov dynamics as a multifocal lens to map multiscale complex networks. arXiv:2211.04287. Retrieved from https://doi.org/10.48550/arXiv.2211.04287
[23] Scott Emmons and Peter J. Mucha. 2019. Map equation with metadata: Varying the role of attributes in community detection. *Physical Review E* 100, 2 (2019), 022301.
[24] Anton Eriksson. 2018. *Interactive Visualization of Community Structure in Complex Networks*. Master's thesis. Department of Physics, Umeå University. diva:1215352 Retrieved from http://urn.kb.se/resolve?urn=urn%3Anbn%3Ase%3Aumu%3Adiva-148551 diva2:1215352.
[25] Anton Eriksson, Timoteo Carletti, Renaud Lambiotte, Alexis Rojas, and Martin Rosvall. 2022. Flow-based community detection in hypergraphs. In *Proceedings of the Higher-Order Systems*. Springer International Publishing, 141–161.
[26] Anton Eriksson, Daniel Edler, Alexis Rojas, Manlio de Domenico, and Martin Rosvall. 2021. How choosing random-walk model and network representation matters for flow-based community detection in hypergraphs. *Communications Physics* 4, 1 (2021), 1–12.
[27] Alcides Viamontes Esquivel and Martin Rosvall. 2012. Comparing network covers using mutual information. arXiv:1202.0425. Retrieved from https://doi.org/10.48550/arXiv.1202.0425
[28] Carmel Farage, Daniel Edler, Anna Eklöf, Martin Rosvall, and Shai Pilosof. 2021. Identifying flow modules in ecological networks using infomap. *Methods in Ecology and Evolution* 12, 5 (2021), 778–786.
[29] Lester R. Ford Jr and Delbert R. Fulkerson. 1956. Maximal flow through a network. *Canadian Journal of Mathematics* 8 (1956), 399–404.
[30] Santo Fortunato. 2010. Community detection in graphs. *Physics Reports* 486, 3 (2010), 75–174.
[31] Santo Fortunato and Marc Barthélemy. 2007. Resolution limit in community detection. *Proceedings of the National Academy of Sciences* 104, 1 (2007), 36–41.
[32] Santo Fortunato and Darko Hric. 2016. Community detection in networks: A user guide. *Physics Reports* 659 (2016), 1–44.
[33] Amir Ghasemian, Homa Hosseinmardi, and Aaron Clauset. 2019. Evaluating overfit and underfit in models of network community structure. *IEEE Transactions on Knowledge and Data Engineering* 9 (2019), 1–1.
[34] David F. Gleich. 2015. PageRank beyond the web. *SIAM Review* 57, 3 (2015), 321–363.
[35] Pablo M. Gleiser and Leon Danon. 2003. Community structure in jazz. *Advances in Complex Systems* 06, 04 (2003), 565–573.
[36] Gene H. Golub and Charles F. Van Loan. 2008. Matrix computations. 3rd edn the johns hopkins university press. *Baltimore, MD* (2008).
[37] Benjamin H. Good, Yves-Alexandre de Montjoye, and Aaron Clauset. 2010. Performance of modularity maximization in practical contexts. *Physical Review E* 81, 4 (2010), 046106.
[38] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. Association for Computing Machinery, 855–864.
[39] Nicolas A. Hazzi, Juan Sebastián Moreno, Carolina Ortiz-Movliav, and Rubén Darío Palacio. 2018. Biogeographic regions and events of isolation and diversification of the endemic biota of the tropical andes. *Proceedings of the National Academy of Sciences* 115, 31 (2018), 7985–7990.
[40] Anton Holmgren, Daniel Edler, and Martin Rosvall. 2018. Infomap Network Navigator. Retrieved from https://mapequation.org/navigator

[41] Anton Holmgren, Daniel Edler, and Martin Rosvall. 2022. Infomap Online. Retrieved from https://mapequation.org/infomap
[42] Anton Holmgren, Daniel Edler, and Martin Rosvall. 2022. The Map Equation Demo. Retrieved from https://mapequation.org/demo
[43] Anton Holmgren, Daniel Edler, and Martin Rosvall. 2022. The MapEquation Alluvial Diagram Generator. Retrieved from https://mapequation.org/alluvial
[44] Anton Holmgren, Daniel Edler, and Martin Rosvall. 2023. Mapping change in higher-order networks with multilevel and overlapping communities. *Applied Network Science* 8, 1 (2023), 42.
[45] David A. Huffman. 1952. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE* 40, 9 (1952), 1098–1101.
[46] Tatsuro Kawamoto and Martin Rosvall. 2015. Estimating the resolution limit of the map equation in community detection. *Physical Review E* 91, 1 (2015), 012809.
[47] Masoumeh Kheirkhahzadeh, Andrea Lancichinetti, and Martin Rosvall. 2016. Efficient community detection of network flows for varying Markov times and bipartite networks. *Physical Review E* 93 (2016), 032309.
[48] Renaud Lambiotte, Jean-Charles Delvenne, and Mauricio Barahona. 2014. Random walks, Markov processes and the multiscale modular organization of complex networks. *IEEE Transactions on Network Science and Engineering* 1 (2014), 76–90.
[49] Renaud Lambiotte and Martin Rosvall. 2012. Ranking and clustering of nodes in networks with smart teleportation. *Physical Review E* 85, 5 (2012), 056107.
[50] Renaud Lambiotte, Martin Rosvall, and Ingo Scholtes. 2019. From networks to optimal higher-order models of complex systems. *Nature Physics* 15, 4 (2019), 313–320.
[51] Andrea Lancichinetti and Santo Fortunato. 2009. Community detection algorithms: A comparative analysis. *Physical Review E* 80, 5 (2009), 056117.
[52] Andrea Lancichinetti, Santo Fortunato, and János Kertész. 2009. Detecting the overlapping and hierarchical community structure in complex networks. *New Journal of Physics* 11, 3 (2009), 033015.
[53] Herman B. Leonard. 1983. Elicitation of honest preferences for the assignment of individuals to positions. *Journal of political Economy* 91, 3 (1983), 461–479.
[54] Maja Lindström, Rohit Sahasrabuddhe, Anton Holmgren, Christopher Blöcker, Tommy Löfstedt, and Martin Rosvall. 2023. Mapping compact memory-biased dynamics reveals overlapping communities. arXiv:2304.05775. Retrieved from https://doi.org/10.48550/arXiv.2304.05775
[55] Leland McInnes, John Healy, Nathaniel Saul, and Lukas Großberger. 2018. UMAP: Uniform manifold approximation and projection. *Journal of Open Source Software* 3, 29 (2018), 861.
[56] Magnus Neuman, Joaquín Calatayud, Viktor Tasselius, and Martin Rosvall. 2024. Module-based regularization improves Gaussian graphical models when observing noisy data. *Applied Network Science* 9, 1 (2024), 6.
[57] Magnus Neuman, Viktor Jonsson, Joaquín Calatayud, and Martin Rosvall. 2022. Cross-validation of correlation networks using modular structure. *Applied Network Science* 7, 1 (2022), 75.
[58] Magnus Neuman, Jelena Smiljanić, and Martin Rosvall. 2025. Reliable data clustering with Bayesian community detection. arXiv:2510.15013. Retrieved from https://doi.org/10.48550/arXiv.2510.15013
[59] Mark E. J. Newman, George T. Cantwell, and Jean-Gabriel Young. 2020. Improved mutual information measure for clustering, classification, and community detection. *Physical Review E* 101, 4 (2020), 042304.
[60] John Palowitch, Shankar Bhamidi, and Andrew B. Nobel. 2018. Significance-based community detection in weighted networks. *Journal of Machine Learning Research* 18, 188 (2018), 1–48.
[61] Leto Peel, Daniel B. Larremore, and Aaron Clauset. 2017. The ground truth about metadata and community detection in networks. *Science Advances* 3, 5 (2017), e1602548.
[62] Tiago P. Peixoto. 2013. Parsimonious module inference in large networks. *Physical Review Letters* 110, 14 (2013), 148701.
[63] Tiago P. Peixoto. 2019. *Bayesian Stochastic Blockmodeling*. John Wiley and Sons, Ltd, Chapter 11.
[64] Christian Persson, Ludvig Bohlin, Daniel Edler, and Martin Rosvall. 2016. Maps of sparse Markov chains efficiently reveal community structure in network flows with memory. arXiv:1606.08328. Retrieved from https://doi.org/10.48550/arXiv.1606.08328
[65] Pascal Pons and Matthieu Latapy. 2006. Computing communities in large networks using random walks. *Journal of Graph Algorithms and Applications* 10 (2006), 191–218.
[66] Jorma Rissanen. 1978. Modeling by shortest data description. *Automatica* 14, 5 (1978), 465–471.
[67] Alexis Rojas, Joaquin Calatayud, Michał Kowalewski, Magnus Neuman, and Martin Rosvall. 2021. A multiscale view of the Phanerozoic fossil record reveals the three major biotic transitions. *Communications Biology* 4, 1 (2021), 309.
[68] Giulio Rossetti, Letizia Milli, and Rémy Cazabet. 2019. CDLIB: A python library to extract, compare and evaluate communities from complex networks. *Applied Network Science* 4, 1 (2019), 1–26.

[69] Martin Rosvall and Carl T. Bergstrom. 2008. Maps of random walks on complex networks reveal community structure. *Proceedings of the National Academy of Sciences* 105, 4 (2008), 1118–1123.
[70] Martin Rosvall and Carl T. Bergstrom. 2010. Mapping change in large networks. *PLoS One* 5, 1 (2010), e8694.
[71] Martin Rosvall and Carl T. Bergstrom. 2011. Multilevel compression of random walks on networks reveals hierarchical organization in large integrated systems. *PLoS One* 6, 4 (2011), e18209.
[72] Martin Rosvall, Alcides V. Esquivel, Andrea Lancichinetti, Jevin D. West, and Renaud Lambiotte. 2014. Memory in network flows and its effects on spreading dynamics and community detection. *Nature Communications* 5, 1 (2014), 4630.
[73] A. I. Saltykov. 1995. The number of components in a random bipartite graph. *Discrete Mathematics and Applications* 5, 6 (1995), 515–524.
[74] Michael T. Schaub, Jean-Charles Delvenne, Martin Rosvall, and Renaud Lambiotte. 2017. The many facets of community detection in complex networks. *Applied Network Science* 2, 1 (2017), 1–13.
[75] Michael T. Schaub, Jean-Charles Delvenne, Sophia N. Yaliraki, and Mauricio Barahona. 2012. Markov dynamics as a zooming lens for multiscale community detection: Non clique-like communities and the field-of-view limit. *PLoS One* 7, 2 (2012), 1–11.
[76] Michael T. Schaub, Renaud Lambiotte, and Mauricio Barahona. 2012. Encoding dynamics for multiscale community detection: Markov time sweeping for the map equation. *Physical Review E* 86, 2 (2012), 026112.
[77] Claude E. Shannon. 1948. A mathematical theory of communication. *Bell System Technical Journal* 27, 3 (1948), 379–423.
[78] Jelena Smiljanić, Daniel Edler, and Martin Rosvall. 2020. Mapping flows on sparse networks with missing links. *Physical Review E* 102, 1 (2020), 012302.
[79] Jelena Smiljanić, Christopher Blöcker, Daniel Edler, and Martin Rosvall. 2021. Mapping flows on weighted and directed networks with incomplete observations. *Journal of Complex Networks* 9, 6 (2021), 1–17.
[80] Fan-Yun Sun, Meng Qu, Jordan Hoffmann, Chin-Wei Huang, and Jian Tang. 2019. vgraph: A generative model for joint community detection and node representation learning. *Advances in Neural Information Processing Systems* 32 (2019), 541–524.
[81] Leo Torres, Ann S. Blevins, Danielle Bassett, and Tina Eliassi-Rad. 2021. The why, how, and when of representations for complex systems. *SIAM Review* 63, 3 (2021), 435–485.
[82] Vincent A. Traag, Ludo Waltman, and Nees Jan van Eck. 2019. From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports* 9, 1 (2019), 5233.
[83] William Vickrey. 1961. Counterspeculation, auctions, and competitive sealed tenders. *The Journal of Finance* 16, 1 (1961), 8–37.
[84] Jaewon Yang and Jure Leskovec. 2014. Overlapping communities explain core-periphery organization of networks. *Proceedings of the IEEE* 102, 12 (2014), 1892–1902.
[85] Wayne W. Zachary. 1977. An information flow model for conflict and fission in small groups. *Journal of Anthropological Research* 33, 4 (1977), 452–473.
[86] Lovro Šubelj, Nees Jan van Eck, and Ludo Waltman. 2016. Clustering scientific publications based on citation relations: A systematic comparison of different methods. *PLoS One* 11, 4 (2016), 1–23.

Received 15 December 2023; revised 17 November 2025; accepted 25 November 2025